

**LABORATÓRIO DE PROGRAMAÇÃO – 2025/1**  
**Prof. Kenia Parente Lopes Mendonça**

**Material de apoio às aulas**

**SUMÁRIO**

|      |                                                                                |    |
|------|--------------------------------------------------------------------------------|----|
| 1    | Revisão de algoritmos.....                                                     | 3  |
| 1.1  | Impressão / saída de dados.....                                                | 3  |
| 1.2  | Variáveis .....                                                                | 3  |
| 1.3  | Operadores Matemáticos .....                                                   | 3  |
| 1.4  | Leitura/Entrada de dados pelo teclado.....                                     | 4  |
| 1.5  | Estrutura de Decisão Condicional: if .. elif .. else .....                     | 6  |
| 1.6  | Estrutura de Repetição (laço) – While .....                                    | 8  |
| 1.7  | Manipulação básica de listas (usando lista como um vetor unidimensional) ..... | 11 |
| 1.8  | Exercícios resolvidos .....                                                    | 13 |
| 1.9  | Exercícios propostos 1 - revisão de Algoritmos .....                           | 14 |
| 2    | Avaliação booleana de variáveis.....                                           | 16 |
| 2.1  | Valores de variáveis e avaliações booleanas .....                              | 16 |
| 2.2  | Códigos para testes .....                                                      | 16 |
| 3    | Estrutura de repetição FOR .....                                               | 21 |
| 3.1  | Percorrendo listas com FOR.....                                                | 21 |
| 3.2  | Exercícios resolvidos .....                                                    | 22 |
| 3.3  | Exercícios propostos 2 .....                                                   | 24 |
| 4    | Manipulação de listas como matrizes .....                                      | 26 |
| 4.1  | Nomenclatura das linhas e colunas das matrizes na lista.....                   | 26 |
| 4.2  | Códigos para testes .....                                                      | 26 |
| 4.3  | Exercícios resolvidos .....                                                    | 28 |
| 4.4  | Exercícios propostos 3 .....                                                   | 30 |
| 5    | Métodos em listas .....                                                        | 32 |
| 5.1  | Códigos para testes .....                                                      | 32 |
| 5.2  | Exercícios propostos 4 .....                                                   | 35 |
| 6    | Manipulação de strings.....                                                    | 36 |
| 6.1  | Métodos de manipulação de strings.....                                         | 36 |
| 6.2  | Códigos para testes .....                                                      | 36 |
| 6.3  | Fatiamento de strings .....                                                    | 39 |
| 6.4  | Exercícios resolvidos .....                                                    | 39 |
| 6.5  | Exercícios propostos 5.....                                                    | 40 |
| 7    | Functions .....                                                                | 42 |
| 7.1  | Estrutura básica .....                                                         | 42 |
| 7.2  | Exemplos de funções.....                                                       | 42 |
| 7.3  | Funções embutidas vs. Funções do usuário.....                                  | 45 |
| 7.5  | Exercícios resolvidos - 1 .....                                                | 46 |
| 7.6  | Argumentos das funções.....                                                    | 47 |
| 7.7  | Exercícios resolvidos - 2 .....                                                | 49 |
| 7.8  | Exercícios propostos 6.....                                                    | 50 |
| 7.9  | Respostas dos exercícios propostos 6.....                                      | 51 |
| 7.10 | Passagem por valor e por referência .....                                      | 53 |
| 7.11 | Testes de mesa – códigos com funções .....                                     | 55 |
| 8    | Dicionários em Python .....                                                    | 57 |
| 8.1  | Conceitos iniciais.....                                                        | 57 |

|        |                                                                  |           |
|--------|------------------------------------------------------------------|-----------|
| 8.2    | Acessando valores nos dicionários.....                           | 57        |
| 8.3    | Alterando valores no dicionário .....                            | 58        |
| 8.4    | Usando o for nos dicionários .....                               | 60        |
| 8.5    | Métodos .items() e .keys() .....                                 | 60        |
| 8.6    | Exercícios resolvidos .....                                      | 60        |
| 9      | Zip, desempacotamento e enumerate .....                          | 62        |
| 9.1    | Comando zip.....                                                 | 62        |
| 9.2    | Desempacotamento de listas e tuplas .....                        | 63        |
| 9.3    | Desempacotamento de dicionários .....                            | 64        |
| 9.4    | Comando enumerate .....                                          | 64        |
| 9.4    | Exercícios propostos .....                                       | 66        |
| 9.5    | Resolução dos exercícios propostos .....                         | 67        |
| 10     | Manipulando arquivos em Python .....                             | 69        |
| 10.1   | Arquivos .txt .....                                              | 69        |
| 10.2   | Exercício resolvido com manipulação de arquivo .txt .....        | 70        |
| 10.3   | Exercícios propostos .....                                       | 73        |
| 10.4   | Resolução dos exercícios propostos .....                         | 73        |
| 11     | Complementos .....                                               | 75        |
| 11.1   | Exemplos com while True + break .....                            | 75        |
| Anexos | .....                                                            | 77        |
|        | <b>A – Resolução dos exercícios propostos 1 (item 1.9) .....</b> | <b>77</b> |
|        | <b>B – Resolução dos exercícios propostos 2 (item 3.3) .....</b> | <b>79</b> |
|        | <b>C – Resolução dos exercícios propostos 3 (item 4.4) .....</b> | <b>80</b> |
|        | <b>D – Resolução dos exercícios propostos 4 (item 5.2) .....</b> | <b>81</b> |
|        | <b>E – Resolução dos exercícios propostos 5 (item 6.5) .....</b> | <b>82</b> |
|        | <b>F – Códigos diversos de revisão para G1.....</b>              | <b>85</b> |
|        | <b>G – Códigos diversos – usos de funções .....</b>              | <b>90</b> |

*Esse material foi elaborado pela **Prof. Me. Kenia Parente Lopes Mendonça** durante o semestre 2025/1, com base nos roteiros de aula desenvolvidos pela **Professora Me. Madianita Marioti Bogo** durante os semestres 2021/1 e 2022/1, quando ministrou a disciplina **LABORATÓRIO DE PROGRAMAÇÃO**.*

# 1 Revisão de algoritmos

**Conteúdo:** Revisão geral de algoritmos: Impressão / saída de dados; variáveis; operadores matemáticos; entrada de dados; estrutura de decisão condicional: if..elif..else; estrutura de repetição while; manipulação básica de listas.

Um algoritmo é uma sequência finita de passos bem definidos utilizados para resolver um problema. No contexto da programação, algoritmos são escritos em linguagens como Python para controlar o fluxo de um programa.

## 1.1 Impressão / saída de dados

**Código 1.0:**

```
print("Hello, World!")
```

**Impressão / Saída:**

Hello, World!

## 1.2 Variáveis

As variáveis armazenam um valor na memória, que pode ser consultado ou alterado posteriormente.

Tipos comuns de variáveis em Python:

- int: números inteiros (ex: 10)
- float: números com ponto decimal (ex: 3.14)
- str: sequência de caracteres (ex: 'Olá')
- bool: valores lógicos (True ou False)

**Código 1.1:** A variável nome recebe o conteúdo Ana

```
nome = "Ana"
print(nome)
```

**Impressão / Saída:**

Ana

**Código 1.2:** o nome da variável escrito entre aspas é um string normal e não uma variável.

```
nome = "Ana"
print("nome")
```

**Impressão / Saída:**

nome

**Código 1.3:** variável **nome** recebe o conteúdo **Ana** e a variável **sobrenome** recebe o conteúdo **Silva**

```
nome = "Ana"
sobrenome = "Silva"
print("Olá, ", nome, sobrenome)
```

**Impressão / Saída:**

Olá, Ana Silva  
Resultado: 12

## 1.3 Operadores Matemáticos

|    |                                       |
|----|---------------------------------------|
| *  | Multiplicação                         |
| /  | Divisão                               |
| +  | Adição                                |
| -  | Subtração                             |
| ** | Exponencial                           |
| // | Parte inteira do resultado da divisão |
| %  | Resto da divisão (mod)                |

#### Código 1.4:

```
numero = 6
print("Número: ", numero)
resultado = numero + 1
print("Resultado: ", resultado)
```

#### Impressão / Saída:

```
Número: 6
Resultado: 7
```

#### Código 1.5:

```
x = 10
resultado = x+2
print("Resultado: ", resultado)
```

#### Impressão / Saída:

```
Resultado: 12
```

#### Código 1.6:

```
x = 10
resultado = x/3
print("Resultado: ", resultado)
print("Resultado: {0:.2f}".format(resultado))
```

#### Impressão / Saída:

```
Resultado: 3.3333333333333335
Resultado: 3.33
```

Os operadores matemáticos obedecem uma precedência matemática (ordem de execução)

1. Parênteses `()`
2. Exponencial `**`
3. Multiplicação `*`, Divisão `/` e o mod `%` → na ordem que aparecerem na expressão
- ↓
4. Adição `+` e Subtração `-` → na ordem que aparecerem na expressão

## 1.4 Leitura/Entrada de dados pelo teclado

**input** é uma função que captura a entrada do usuário via teclado e retorna uma string com o valor digitado.

#### Código 1.7:

```
nome = input("Digite o nome: ")
print("Nome: ", nome)
```



### Impressão / Saída:

```
Digite o nome: |
```

A função fica à espera da digitação pelo usuário de algo que será retornado como uma string e armazenado na variável **nome**.

### Saída caso seja digitado o nome "Ana":

```
Digite o nome: Ana  
Nome: Ana
```

### Código 1.8: entrada de valores inteiros

```
a = int(input("Digite o valor de a: "))  
b = int(input("Digite o valor de b: "))  
print (a+b)
```

### Impressão / Saída (para o caso de o usuário digitar 5 para o valor de a e 6 para o valor de b):

```
Digite o valor de a: 5  
Digite o valor de b: 6  
11
```

### Código 1.9: entrada de valores inteiros

```
a = int(input("Digite o valor de a: "))  
b = int(input("Digite o valor de b: "))  
soma = a + b  
print (soma)
```

### Impressão / Saída (para o caso de o usuário digitar -5 para o valor de a e -6 para o valor de b):

```
Digite o valor de a: -5  
Digite o valor de b: -6  
-11
```

### Código 1.10: entrada de valores reais

```
x = float(input("Digite o valor de a: "))  
y = float(input("Digite o valor de b: "))  
print (x+y)
```

### Impressão / Saída (para o caso do usuário digitar 3.5 para o valor de x e 2.5 para o valor de y):

```
Digite o valor de a: 3.5  
Digite o valor de b: 2.5  
6.0
```

### Código 1.11: script que lê a base e a altura de um triângulo e apresenta a área do mesmo

```
base = float (input('Digite o valor da base: '))  
alt = float (input('Digite o valor da altura: '))  
area = base*alt/2  
print('Área:', area)
```

### Impressão / Saída (para o caso do usuário digitar 3 para a base e 4 para a altura do triângulo):

```
Digite o valor da base: 3  
Digite o valor da altura: 4  
Área: 6.0
```

## 1.5 Estrutura de Decisão Condicional: if .. elif .. else

A estrutura de decisão condicional permite que um programa execute diferentes blocos de código com base em condições específicas. O Python utiliza as palavras-chave **if**, **elif** e **else** para definir essas condições.

- **if**: Verifica uma condição e executa o bloco de código se for verdadeira.
- **elif**: Define condições adicionais caso a primeira seja falsa.
- **else**: Define um bloco de código que será executado se todas as condições anteriores forem falsas.

### Operadores Relacionais:

`==` : igual a

`!=` : diferente de

`>` : maior que

`<` : menor que

`>=` : maior ou igual a

`<=` : menor ou igual a

### Operadores Lógicos:

- **and**: todas as condições devem ser verdadeiras
- **or**: pelo menos uma condição deve ser verdadeira
- **not**: inverte o valor lógico

**Código 1.12:** o código a seguir recebe um número e informa se ele é positivo, negativo ou zero

```
num = int(input("Digite um número: "))

if num > 0:
    print("O número é positivo.")
elif num < 0:
    print("O número é negativo.")
else:
    print("O número é zero.")
```

**Impressão / Saída** (para o caso do usuário digitar -5 para o número):

```
Digite um número: -5
O número é negativo.
```

**Código 1.13:** o programa a seguir recebe a idade de uma pessoa e informa se ela é maior ou menor de idade.

```
idade = int(input('Digite a idade: '))

if idade >= 18:
    print('Maior de idade!')
else:
    print('Menor de idade!')
```

**Impressão / Saída** (para o caso do usuário digitar 16 para a idade):

```
Digite a idade: 16
Menor de idade!
```

**Código 1.14:** No código a seguir, existem três possibilidades de saída, de acordo com o conteúdo que o usuário digitar para a variável **sexo**.

```
sexo = input('Digite o sexo: ')

if sexo == 'f':
    print('Feminino')
elif sexo == 'm':
    print('Masculino')
else:
    print('Digite um sexo válido')
```

**Impressão / Saída** (para o caso do usuário digitar “m” para o sexo):

```
Digite o sexo: m
Masculino
```

**Código 1.15:** Programa que lê a idade de uma pessoa e apresenta a categoria da mesma, de acordo com a tabela a seguir:

| Idade            | Categoria   |
|------------------|-------------|
| Até 10 anos      | Categoria A |
| 11 até 20 anos   | Categoria B |
| Acima de 20 anos | Categoria C |

```
idade = int(input("Digite a idade: "))
if idade <= 10:
    print("Categoria A")
elif idade <= 20:
    print("Categoria B")
else:
    print("Categoria C")
```

**Impressão / Saída** (para o caso do usuário digitar 18 para a idade):

```
Digite a idade: 18
Categoria B
```

**Código 1.16:** Programa que solicita ao usuário a informação se uma pessoa tem deficiência física (sim ou não) e a idade da pessoa. Depois, informa se ela terá entrada liberada em um evento ou se terá

que pagar ingresso, sendo que terão entrada liberada as pessoas com deficiência física e os idosos (pessoas com 65 anos ou mais).

```
deficiente = input("Tem alguma deficiência física (sim ou não)? ")
idade = int(input("Idade: "))
if deficiente == "sim" or idade >= 65:
    print("Entrada liberada!")
else:
    print("Deve comprar ingresso!")
```

**Impressão / Saída** (para o caso do usuário digitar “não” para a pergunta “Tem alguma deficiência” e 60 para a idade):

```
Tem alguma deficiência física (sim ou não)? não
Idade: 60
Deve comprar ingresso!
```

**Código 1.17:** Programa que solicita ao usuário a profissão e o estado onde mora determinada pessoa. Depois, informa se essa pessoa receberá crédito para comprar livros no Salão do Livro. Ganharão crédito professores ou estudantes do estado do Tocantins.

```
prof = input("Profissão: ")
est = input("Estado: ")
if (prof=="professor" or prof=="estudante") and est=="Tocantins":
    print("Ganhará crédito!")
else:
    print("Não ganhará crédito!")
```

**Impressão / Saída** (para o caso de ser um professor do estado de Goiás):

```
Profissão: professor
Estado: Goiás
Não ganhará crédito!
```

**Código 1.18:** Programa que solicita o sexo (feminino ou masculino) e o turno (matutino ou vespertino) e retorna uma das mensagens: “Bom dia, querida”, “Bom dia, querido”, “Boa tarde, querida” ou “Boa tarde, querido”

```
sexo = input("Digite o sexo (F ou M): ")
turno = input("Digite o turno (M ou V): ")
if turno == "M":
    msg = "Bom dia, querid"
else:
    msg = "Boa tarde, querid"
if sexo == "F":
    msg = msg + "a"
else:
    msg = msg + "o"
print(msg)
```

**Impressão / Saída** (para o caso do usuário digitar F para sexo e M para turno):

```
Digite o sexo (F ou M): F
Digite o turno (M ou V): M
Bom dia, querida
```

## 1.6 Estrutura de Repetição (laço) – While

- No comando de repetição **while**, enquanto a condição for verdadeira, os comandos serão executados; quando for falsa, o laço é finalizado.

Obs: O teste da condição é realizado no início da repetição, assim, se na primeira vez a condição for falsa, as instruções nunca serão executadas.

**Código 1.18:** Imprime os números de 0 (zero) a 5

```
cont = 0
while cont < 6:
    print(cont)
    cont = cont + 1
```

```
cont = 0
while cont <= 5:
    print(cont)
    cont = cont + 1
```

**Impressão / Saída:**

```
0
1
2
3
4
5
```

**Código 1.19:** Lê a quantidade e o preço unitário de 5 produtos comprados por um cliente e apresenta o valor gasto com cada produto.

```
cont = 0
while cont < 5:
    cont = cont + 1
    qt = int (input("Quantidade:"))
    preco = float (input("Preço unitário:"))
    print("Valor gasto com o produto", cont, "foi", qt*preco)
```

**Impressão / Saída** (ao ser digitadas as seguintes quantidades e preços: 2xR\$10,00; 3xR\$5,00; 1xR\$50,00; 4xR\$30,00 e 1xR\$70,00):

```
Quantidade:2
Preço unitário:10
Valor gasto com o produto 1 foi 20.0
Quantidade:3
Preço unitário:5
Valor gasto com o produto 2 foi 15.0
Quantidade:1
Preço unitário:50
Valor gasto com o produto 3 foi 50.0
Quantidade:4
Preço unitário:30
Valor gasto com o produto 4 foi 120.0
Quantidade:1
Preço unitário:70
Valor gasto com o produto 5 foi 70.0
```

**Código 1.20:** acrescenta-se ao código 19 o cálculo da quantidade de produtos comprados que tem o preço maior que R\$ 50,00.



```

cont = 0
cont_mais50 = 0
while cont < 5:
    cont = cont + 1
    qt = int(input("Quantidade:"))
    preco = float(input("Preço unitário:"))
    print("Valor gasto com o produto", cont, "foi", qt*preco)
    if preco>50:
        cont_mais50 = cont_mais50 + 1
print("Quant. de produtos que custam mais que R$50.00: ", cont_mais50)

```

**Impressão / Saída** (ao ser digitadas as seguintes quantidades e preços: 2xR\$10,00; 3xR\$5,00; 1xR\$50,00; 4xR\$30,00 e 1xR\$70,00):

```

Quantidade:2
Preço unitário:10
Valor gasto com o produto 1 foi 20.0
Quantidade:3
Preço unitário:5
Valor gasto com o produto 2 foi 15.0
Quantidade:1
Preço unitário:50
Valor gasto com o produto 3 foi 50.0
Quantidade:4
Preço unitário:30
Valor gasto com o produto 4 foi 120.0
Quantidade:1
Preço unitário:70
Valor gasto com o produto 5 foi 70.0
Quant. de produtos que custam mais que R$50.00:  1

```

**Código 1.21:** programa que lê a idade e o sexo de **um número indeterminado de pessoas**, sendo que a leitura é **encerrada quando o usuário digitar zero para a idade**. Calcula e apresenta a média das idades.

```

soma_idades = 0
qt_pessoas = 0

idade = int(input('Digite a idade:'))
while idade>0:
    sexo = input('Sexo: ')
    soma_idades = soma_idades + idade
    qt_pessoas = qt_pessoas + 1

    idade = int(input('Digite a idade:'))

print('Média das idades:', soma_idades/qt_pessoas)

```

**Impressão / Saída** (ao serem digitadas as seguintes idades e sexo: 18,F; 20,M; 25,M e 20,F)

```

Digite a idade:18
Sexo: F
Digite a idade:20
Sexo: M
Digite a idade:25
Sexo: M
Digite a idade:20
Sexo: F
Digite a idade:0
Média das idades: 20.75

```

**Código 1.22:** acrescenta-se ao código 21 a informação ao final do sexo da pessoa mais nova, considerando que não houve empate.

```
soma_idades = 0
qt_pessoas = 0
menor_idade = 150

idade = int(input('Digite a idade:'))
while idade>0:
    sexo = input('Sexo: ')

    soma_idades = soma_idades + idade
    qt_pessoas = qt_pessoas + 1

    if idade < menor_idade:
        menor_idade = idade
        sexo_nova = sexo

    idade = int(input('Digite a idade:'))

print('Média das idades:', soma_idades/qt_pessoas)
print('Sexo da pessoa mais nova:', sexo_nova)
```

**Impressão / Saída** (ao serem digitas as seguintes idades e sexo: 18,F; 20,M; 25,M e 20,F):

```
Digite a idade:18
Sexo: F
Digite a idade:20
Sexo: M
Digite a idade:25
Sexo: M
Digite a idade:20
Sexo: F
Digite a idade:0
Média das idades: 20.75
Sexo da pessoa mais nova: F
```

## 1.7 Manipulação básica de listas (usando lista como um vetor unidimensional)

Uma lista (list) é uma coleção ou sequência de valores. Os valores que compõem a lista são chamados elementos ou itens, os quais são separados por vírgulas

**Código 1.23:**

```
lista_nomes = ["Ana", "Beto", "Paulo"]
print(lista_nomes)
```

**Impressão / Saída:**

```
['Ana', 'Beto', 'Paulo']
```

**Código 1.24:**

```
numeros = [10, 3, -8]
print(numeros)
```

**Impressão / Saída:**

```
[10, 3, -8]
```

**Código 1.25:** Verificando o tamanho (length) da lista (quantos elementos existem na mesma).

```
lista_nomes = ["Ana", "Beto", "Paulo"]
print("Tamanho da lista: ", len(lista_nomes))
```

Impressão / Saída:

Tamanho da lista: 3

**Código 1.26A e 1.26B:** Verificando se existe um elemento na lista.

```
lista_nomes = ["Ana", "Beto", "Paulo"]

if "Ana" in lista_nomes:
    print("Existe na lista!")
else:
    print("Não existe na lista!")
```

Impressão / Saída:

Existe na lista!

```
lista_nomes = ["Ana", "Beto", "Paulo"]

if "João" in lista_nomes:
    print("Existe na lista!")
else:
    print("Não existe na lista!")
```

Impressão / Saída:

Não existe na lista!

**Código 1.27:** Cada valor na lista é identificado por um índice (posição na lista).

```
lista_nomes = ["Ana", "Beto", "Paulo"]
print("Nome: ", lista_nomes[0])
print("Nome: ", lista_nomes[1])
print("Nome: ", lista_nomes[2])
```

Impressão / Saída:

Nome: Ana  
Nome: Beto  
Nome: Paulo

**Código 1.28:** Percorrendo a lista com o laço **while**:

```
lista_nomes = ["Ana", "Beto", "Paulo"]
cont = 0

while cont < len(lista_nomes):
    print("Posição:", cont, "Nome: ", lista_nomes[cont])
    cont = cont + 1
```

Impressão / Saída:

Posição: 0 Nome: Ana  
Posição: 1 Nome: Beto  
Posição: 2 Nome: Paulo

**Código 1.29:** Inserindo elemento no final da lista:



```

lista_nomes = []
print('Tamanho da lista:', len(lista_nomes), 'Elementos: ', lista_nomes)

lista_nomes.append('Maria')
print('Tamanho da lista:', len(lista_nomes), 'Elementos: ', lista_nomes)

lista_nomes.append('João')
print('Tamanho da lista:', len(lista_nomes), 'Elementos: ', lista_nomes)

```

**Impressão / Saída:**

```

Tamanho da lista: 0 Elementos: []
Tamanho da lista: 1 Elementos: ['Maria']
Tamanho da lista: 2 Elementos: ['Maria', 'João']

```

**Código 1.30:** Alterando o valor de um elemento da lista.

```

lista_nomes = ["Ana", "Beto", "Paulo"]
print(lista_nomes)

lista_nomes[0]="Ana Paula"
print(lista_nomes)

```

**Impressão / Saída:**

```

['Ana', 'Beto', 'Paulo']
['Ana Paula', 'Beto', 'Paulo']

```

**Código 1.31:** Criando uma lista vazia e inserindo 4 elementos digitados pelo usuário.

```

lista_nomes = []
cont = 0

while cont < 4:
    lista_nomes.append(input("Digite um nome: "))
    print("Tamanho:", len(lista_nomes), " Elementos:", lista_nomes)
    cont=cont+1

```

**Impressão / Saída:**

```

Digite um nome: Ana
Tamanho: 1 Elementos: ['Ana']
Digite um nome: Maria
Tamanho: 2 Elementos: ['Ana', 'Maria']
Digite um nome: Paulo
Tamanho: 3 Elementos: ['Ana', 'Maria', 'Paulo']
Digite um nome: João
Tamanho: 4 Elementos: ['Ana', 'Maria', 'Paulo', 'João']

```

## 1.8 Exercícios resolvidos

1. Atualmente, ano de 2025, o Museu A tem 1000 obras em sua coleção e o Museu B tem 500 obras. Todo ano o Museu A consegue adquirir 100 obras e o museu B consegue adquirir 130 obras. Calcule e informe em que ano o museu B terá mais obras que o museu A.

```

ano = 2025
museuA = 1000
museuB = 500
while museuA <= museuB:
    museuA += 100 #museuA = museuA + 200
    museuB += 130
    ano += 1

print(f"O ano em que o museu B terá mais obras que o museu A será {ano}")

```

2. Seja a lista de números [10, 25, 32, 44, 51, 63, 72, 80]. Elabore um código que conte e apresente quantos números pares há nessa lista.

```

numeros = [10, 25, 32, 44, 51, 63, 72, 80]
cont_pares = 0
i = 0
while i < len(numeros):
    if numeros[i] % 2 == 0:
        cont_pares += 1
    i += 1
print("Há {} números pares nessa lista".format(cont_pares))

```

## 1.9 Exercícios propostos 1 - revisão de Algoritmos

- Escreva um programa que solicite ao usuário seu nome e idade e, em seguida, exiba a mensagem:  
"Olá, [nome]! Você tem [idade] anos."
- Crie um programa que pergunte o nome e a profissão do usuário e exiba uma mensagem no seguinte formato:  
"Olá, [nome]! Você trabalha como [profissão]. Desejamos um ótimo dia!"
- Peça ao usuário dois números inteiros e exiba a soma, subtração, multiplicação, divisão e o resto da divisão entre eles.
- Escreva um programa que peça ao usuário a temperatura em graus Celsius e a converta para Fahrenheit utilizando a fórmula:  

$$F = ^\circ C \times \frac{9}{5} + 32$$
- Escreva um programa que solicite um número ao usuário e informe se ele é positivo, negativo ou zero.
- Peça ao usuário um número inteiro e informe se ele é par ou ímpar.
- Escreva um programa que peça ao usuário um número inteiro positivo (entre 5 e 50) e exiba na tela todos os números de 0 até esse número.
- Peça ao usuário que digite um número N (entre 100 e 500) e calcule a soma de todos os números pares de 1 até N.
- Peça ao usuário para digitar a temperatura atual e classifique-a de acordo com os critérios abaixo:

- Abaixo de 15°C → "Frio"
  - Entre 15°C e 25°C → "Agradável"
  - Acima de 25°C → "Calor"
10. Crie um programa que peça ao usuário para digitar cinco nomes e os armazene em uma lista. Depois, exiba os nomes digitados um por um.
  11. Peça ao usuário para cadastrar o nome e o preço de três produtos em uma lista. Depois, exiba os produtos cadastrados no seguinte formato.  
Produto: [nome] – Preço: R\$ [preço]
  12. Dada a lista [['A', 'B'], ['C'], ['D', 'E', 'F']], escreva um código que complete todas as sublistas com 'X' até que todas tenham o mesmo tamanho da maior sublista.

## 2 Avaliação booleana de variáveis

### 2.1 Valores de variáveis e avaliações booleanas

| Valor             | Avaliação booleana |
|-------------------|--------------------|
| "" (string vazia) | <i>False</i>       |
| [] (lista vazia)  | <i>False</i>       |
| 0 (zero)          | <i>False</i>       |
| None              | <i>False</i>       |
| False             | <i>False</i>       |

### 2.2 Códigos para testes

#### Código 2.1:

```
a = "oi"
if a:
    print("1 -", a)

a = ""
if not a:
    print("2 -", a)
```

Impressão / Saída:

1 - oi

#### Código 2.2:

```
a = ''
if a:
    print("3-", a)

a = ''
if not a:
    print("4-", a)
```

Impressão / Saída:

4-

#### Código 2.3:

```
a = []
if a:
    print("5-", a)

a = []
if not a:
    print("6-", a)
```

Impressão / Saída:

6- []

#### Código 2.4:

```
a=20
if a:
    print("7-", a)

a=20
if not a:
    print("8-", a)
```

Impressão / Saída:

7- 20

#### Código 2.5:

```
a=None
if a:
    print("9-", a)

a=None
if not a:
    print("10-", a)

a="None"
if a:
    print("11-", a)

a="None"
if not a:
    print("12-", a)
```

Impressão / Saída:

10- None

11- None

#### Código 2.6:

```
a=False
if a:
    print("13-", a)

a=False
if not a:
    print("14-", a)
```

Impressão / Saída:

14- False

#### Código 2.7:

```
a=True
if a:
    print("15-", a)

a=True
if not a:
    print("16-", a)
```

Impressão / Saída:

15- True

Código 2.8:

```
a = 20
b = 5
if a and b < 10:
    print("Verdade")
else:
    print("Falso")
```

Impressão / Saída:

Verdade

Código 2.9:

```
a = 0
if a and b < 10:
    print("Verdade")
else:
    print("Falso")
```

Impressão / Saída:

Falso

Código 2.10:

```
a = 1
b = 5
if a < 10 and b < 10:
    print("Verdade")
else:
    print("Falso")
```

Impressão / Saída:

Verdade

Código 2.11:

```
a = "João"
b = "Ana"
if a and b == "Ana":
    print("Verdadeiro")
else:
    print("Falso")
```

Impressão / Saída:

Verdadeiro

Código 2.12:

```
a = "João"
b = "Ana"
if a == "Ana" and b == "Ana":
    print("Verdadeiro")
else:
    print("Falso")
```

Impressão / Saída:

Falso



Código 2.13:

```
a = "Ana"
b = "Ana"
if a or b == "Ana":
    print("Verdadeiro1")
else:
    print("Falso1")
```

Impressão / Saída:

Verdadeiro1

Código 2.14:

```
a = "João"
b= "Pedro"
if a or b == "Ana":
    print("Verdadeiro2")
else:
    print("Falso2")
```

Impressão / Saída:

Verdadeiro2

Código 2.15:

```
a = "João"
b= "Pedro"
if a == "Ana" or b == "Ana":
    print("Verdadeiro3")
else:
    print("Falso3")
```

Impressão / Saída:

Falso3

### and

|                 |       |
|-----------------|-------|
| True and True   | True  |
| True and False  | False |
| False and True  | False |
| False and False | False |

O operador lógico and retorna o primeiro valor falso encontrado na expressão. Se todos os valores forem verdadeiros, retorna o último valor.

### or

|                |       |
|----------------|-------|
| True or True   | True  |
| True or False  | True  |
| False or True  | True  |
| False or False | False |

O operador lógico or retorna o primeiro valor verdadeiro encontrado na expressão. Se todos os valores forem falsos, retorna o último valor.

**Código 2.16:**

```
a=5
b=6
print ("a and b:", a and b)
print ("b and a:", b and a)
print ("b and a and 0:", b and a and 0)
print ("0 and a and b:", 0 and a and b)
```

**Impressão / Saída:**

```
a and b: 6
b and a: 5
b and a and 0: 0
0 and a and b: 0
```

**Código 2.17:**

```
a=5
b=6
print ("a or b:", a or b)
print ("b or a:", b or a)
print ("b or a or 0:", b or a or 0)
print ("0 or a or b:", 0 or a or b)
```

**Impressão / Saída:**

```
a or b: 5
b or a: 6
b or a or 0: 6
0 or a or b: 5
```

## 3 Estrutura de repetição FOR

### 3.1 Percorrendo listas com FOR

```
range(start, stop, step)
```

↓  
esse valor nunca  
é incluído na  
sequência

- Ao especificar somente **um valor**, a função **range** retorna uma lista de número (no **for** a variável vale um número da lista a cada iteração), em um intervalo que se inicia com zero e termina no número inteiro que antecede o número especificado.

Código 3.1:

```
for num in range(5):  
    print(num)
```

Impressão / Saída:

```
0  
1  
2  
3  
4
```

- Ao especificar **dois valores**, a função **range** retorna uma lista de números, em um intervalo que se inicia com o primeiro número especificado e termina no número inteiro que antecede o segundo número especificado.

Código 3.2:

```
for num in range(10,15):  
    print(num)
```

Impressão / Saída:

```
10  
11  
12  
13  
14
```

- Ao especificar **três valores**, a função **range** retorna uma lista de números, em um intervalo que se inicia com o primeiro número especificado e termina no número inteiro mais próximo que antecede o segundo número especificado, **incrementando** o terceiro valor especificado.

Código 3.3:

```
for num in range(10,20,2):  
    print(num)
```

Impressão / Saída:

```
10  
12  
14  
16  
18
```

#### Código 3.4:

```
for num in range(15,10,-1):  
    print(num)
```

Impressão / Saída:

```
15  
14  
13  
12  
11
```

### 3.2 Exercícios resolvidos

1. Escreva um programa que leia 10 números reais e guarde em uma lista, e:
  - a) imprima a lista completa;
  - b) informe a quantidade de elementos negativos;
  - c) informe a soma dos elementos positivos.

```
qt_negativos = 0  
soma_pos = 0  
numeros=[]  
for i in range(10):  
    num=float(input('Digite um número: '))  
    numeros.append(num)  
    if numeros[i]<0:  
        qt_negativos+=1  
    else:  
        soma_pos+=numeros[i]  
  
print('lista digitada:',numeros)  
print('Qtde de negativos: ',qt_negativos)  
print('Soma dos elementos positivos: ',soma_pos)
```

2. Incremente o código anterior com as seguintes instruções:
  - a) troque os 5 primeiros elementos da lista pelo número 10;
  - b) dobre o valor dos últimos 5 elementos da lista;
  - c) imprima a lista novamente.

```
qt_negativos = 0  
soma_pos = 0  
numeros=[]  
for i in range(10):  
    num=float(input('Digite um número: '))  
    numeros.append(num)  
    if numeros[i]<0:  
        qt_negativos+=1  
    else:  
        soma_pos+=numeros[i]  
  
print('lista digitada:',numeros)  
print('Qtde de negativos: ',qt_negativos)  
print('Soma dos elementos positivos:',soma_pos)  
  
for i in range(10):  
    if i <5:  
        numeros[i] = 10  
    else:  
        numeros[i] *= 2  
  
print('Nova lista: ',numeros)
```

3. Considerando a lista `peessoas = [ ['Ana Maria', 10], ['Pedro', 20], ['Carlos', 8], ['Carla', 18] ]`

Faça um programa que realize as seguintes ações, sendo que ele deve funcionar se a lista for preenchida com outros dados e tenha outro tamanho:

- a. imprima a lista criada, de acordo com o seguinte modelo:

Nome: Ana Maria Idade: 10

Nome: Pedro Idade: 20

...

```
peessoas = [ ["Ana Maria",10] ,
              ["Pedro",20] ,
              ["Carlos",8] ,
              ["Carla",18]]

for i in range(len(peessoas)):
    print("Nome:",peessoas[i][0],"Idade:",peessoas[i][1])
```

- b. imprima uma relação, de acordo com o seguinte modelo:

Nome: Ana Maria é menor de idade

Nome: Pedro é maior de idade

...

```
for i in range(0,len(peessoas)):
    if pessoas[i][1]<18:
        print(peessoas[i][0],"é menor de idade")
    else:
        print(peessoas[i][0],"é maior de idade")
```

- c. solicite o nome de uma pessoa e informe a idade digitada ou a informação que a pessoa não foi encontrada.

```
encontrada = False
nome = input("Qual o nome a ser pesquisado?")
for i in range(len(peessoas)):
    if pessoas[i][0]==nome:
        print("Idade de",nome,peessoas[i][1])
        encontrada = True
if not encontrada:
    print("Pessoa não cadastrada")
```

4. O trecho de código a seguir cria e imprime uma lista que representa o cardápio de um food truck.

|   | Produto         | Preço |
|---|-----------------|-------|
| 0 | Cachorro quente | 5.00  |
| 1 | X-Salada        | 10.00 |
| 2 | X-Bacon         | 12.00 |
| 3 | Bauru           | 8.00  |
| 4 | Refrigerante    | 4.00  |
| 5 | Suco            | 6.00  |

```
cardapio = [['Cachorro-quente',5.00],
            ['X-Salada', 10.00],
            ['X-Bacon', 12.00],
            ['Bauru', 8.00],
            ['Refrigerante', 4.00],
            ['Suco', 6.00]]
```

```
print(cardapio)
```

Continue o programa de forma que, após a criação da lista, leia o código de um item e a quantidade deste item para os pedidos de uma mesa em uma lanchonete. A cada pedido realizado é perguntado

se a conta deve ser fechada e, caso o usuário digite sim (S), os pedidos devem ser encerrados e deve ser apresentado o valor da conta.

| EXEMPLO                   |            |
|---------------------------|------------|
| Entrada                   |            |
| Código                    | Quantidade |
| 0                         | 2          |
| 3                         | 1          |
| 4                         | 1          |
| Saída                     |            |
| Valor da conta: R\$ 22.00 |            |

```
cardapio = [ ["Cachorro-quente", 5.00],
              ["X-Salada", 10.00],
              ['X-Bacon', 12.00],
              ['Bauru', 8.00],
              ['Refrigerante', 4.00],
              ['Suco', 6.00]]

print(cardapio)

conta = 0
fechar="N"
while fechar == "N":
    codigo = int(input("Código do item: "))
    qtde = int(input("Qual a qtde desse item? "))
    preco = cardapio[codigo][1]
    conta = preco*qtde
    fechar=input("Deseja fechar a conta(S) ou (N)?")

print('O valor da conta é: R$ {:.2f}'.format(conta))
```

### 3.3 Exercícios propostos 2

1. Considerando a lista a seguir:

`[['A', 'B', 'C', 'D', 'E'], ['P', 'Q', 'R', 'S'], ['X', 'Y', 'Z']]`

Altere a última letra de cada sublista pela primeira, sendo que o programa deve funcionar com outro conteúdo na lista.

Para a lista em questão a saída será:

`[['A', 'B', 'C', 'D', 'A'], ['P', 'Q', 'R', 'P'], ['X', 'Y', 'X']]`

2. Imprima os nomes da lista a seguir em ordem alfabética, decrescente, com um nome em cada linha.  
`lista = ['Joao', 'Pedro', 'Zezé', 'Daniel', 'Ana', 'Maria', 'Carla']`

3. Remova todas as ocorrências do nome 'Ana' da lista a seguir:

`lista = ['Joao', 'Ana', 'Zezé', 'Daniel', 'Ana', 'Maria', 'Carla']`, e crie uma nova lista sem o nome Ana.

4. Faça o código acima removendo o nome “Ana”, mas sem criar uma nova lista.

5. Considerando a lista a seguir:

```
[[ 'A', 'B', 'C' ], [ 'I', 'J' ], [ 'P', 'Q', 'R', 'S', 'T' ], [ 'X', 'Y', 'Z' ]]
```

Altere a lista, preenchendo com 'X' as letras que faltam em cada sublista para que todas elas tenham o tamanho da maior sublista.

Para a lista em questão a saída será:

```
[[ 'A', 'B', 'C', 'X', 'X' ], [ 'I', 'J', 'X', 'X', 'X' ], [ 'P', 'Q', 'R', 'S', 'T' ], [ 'X', 'Y', 'Z', 'X', 'X' ]]
```



## 4 Manipulação de listas como matrizes

### 4.1 Nomenclatura das linhas e colunas das matrizes na lista

Exemplo de uma matriz 2x3: (2 linhas e 3 colunas)

|     |   |   |   |   |               |                                     |
|-----|---|---|---|---|---------------|-------------------------------------|
|     |   | 0 | 1 | 2 |               |                                     |
|     |   |   |   |   | $\Rightarrow$ | $\text{mat}[0] \rightarrow [3,6,5]$ |
|     |   |   |   |   |               | $\text{mat}[1] \rightarrow [4,1,2]$ |
|     |   |   |   |   |               | $\text{mat}[0][0] \rightarrow 3$    |
|     |   |   |   |   |               | $\text{mat}[0][1] \rightarrow 6$    |
|     |   |   |   |   |               | $\text{mat}[0][2] \rightarrow 5$    |
|     |   |   |   |   |               | $\text{mat}[1][0] \rightarrow 4$    |
|     |   |   |   |   |               | $\text{mat}[1][1] \rightarrow 1$    |
|     |   |   |   |   |               | $\text{mat}[1][2] \rightarrow 2$    |
| mat | 0 | 3 | 6 | 5 |               |                                     |
|     | 1 | 4 | 1 | 2 |               |                                     |

### 4.2 Códigos para testes

Código 4.1:

```
mat = []
for i in range(3):
    nova_linha = []
    for j in range(3):
        num = int(input("mat["+str(i)+"]["+str(j)+"]: "))
        nova_linha.append(num)
    mat.append(nova_linha)

print(mat)
```

**Impressão / Saída:** no caso do usuário digitar os números de 1 a 9 para os elementos da matriz.

```
mat[0][0]: 1
mat[0][1]: 2
mat[0][2]: 3
mat[1][0]: 4
mat[1][1]: 5
mat[1][2]: 6
mat[2][0]: 7
mat[2][1]: 8
mat[2][2]: 9
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

**Código 4.2:** impressão linha a linha acrescentando ao código as seguintes linhas.

```
print("*****")

for i in range(3):
    print(mat[i])
```

**Impressão / Saída:**

```
*****
[1, 2, 3]
[4, 5, 6]
[7, 8, 9]
```

**Código 4.3:** impressão dessa matriz em formato de matriz

```
str_mat = ""
for i in range(3):
    for j in range(3):
        str_mat += str(mat[i][j])+"\t"
    str_mat += "\n"

print(str_mat)
```

Impressão / Saída:

```
****
1      2      3
4      5      6
7      8      9
```

**Código 4.4:** código completo de criação de uma matriz 3 x 3 e sua impressão

```
mat = []
for i in range(3):
    nova_linha = []
    for j in range(3):
        num = int(input("mat["+str(i)+"]["+str(j)+"]: "))
        nova_linha.append(num)
    mat.append(nova_linha)

print(mat)
print("*****")

for i in range(3):
    print(mat[i])

print("*****")

str_mat = ""
for i in range(3):
    for j in range(3):
        str_mat += str(mat[i][j])+"\t"
    str_mat += "\n"

print(str_mat)
```

Impressão / Saída:

```
mat[0][0]: 2
mat[0][1]: 5
mat[0][2]: 7
mat[1][0]: -8
mat[1][1]: -3
mat[1][2]: -4
mat[2][0]: 0
mat[2][1]: 8
mat[2][2]: 9
[[2, 5, 7], [-8, -3, -4], [0, 8, 9]]
*****
[2, 5, 7]
[-8, -3, -4]
[0, 8, 9]
****
2      5      7
-8     -3     -4
0      8      9
```

### 4.3 Exercícios resolvidos

1. Imprima a matriz resultante da digitação dos seus elementos pelo usuário, sendo que inicialmente o usuário deverá informar quantas linhas e quantas colunas a matriz terá.

```
mat = []
linhas = int(input("Quantas linhas terá a matriz? "))
colunas = int(input("Quantas colunas terá a matriz? "))

for i in range(linhas):
    nova_linha = []
    for j in range(colunas):
        num = int(input("linha "+str(i)+", coluna "+str(j)+": "))
        nova_linha.append(num)
    mat.append(nova_linha)

str_mat = ""
for i in range(linhas):
    for j in range(colunas):
        str_mat += str(mat[i][j])+"\\t"
    str_mat += "\\n"

print(str_mat)
```

2. O programa abaixo preenche e imprime uma matriz 4 x 4 com números inteiros variando entre 0 e 5, inclusive:

```
import random
mat = []
for i in range(4):
    nova_linha = []
    for j in range(4):
        nova_linha.append(random.randint(0,5))
    mat.append(nova_linha)
print(mat[i])
```

Após o preenchimento, considerando os conceitos de matrizes, apresente:

- a) os elementos da primeira linha
  - b) os elementos da última linha
  - c) os elementos da diagonal principal
  - d) os elementos da diagonal secundária
  - e) informação se é uma matriz triangular inferior ou se não é uma matriz triangular inferior
- Obs Matriz triangular inferior: ocorre quando todos os termos que estão acima da diagonal principal são iguais a zero.

```

import random
mat = []
for i in range(4):
    nova_linha = []
    for j in range(4):
        nova_linha.append(random.randint(0,5))
    mat.append(nova_linha)
    print(mat[i])

print("1ª linha: ", mat[0])
print("Última linha: ", mat[3])

principal = []
secundaria = []
triang_inferior = True

for i in range(4):
    for j in range(4):
        if i==j:
            principal.append(mat[i][j])
        if i + j == 3:
            secundaria.append(mat[i][j])
        if i<j and mat[i][j]!=0:
            triang_inferior = False
if triang_inferior:
    print("É triangular inferior")
else:
    print("Não é triangular inferior")
sec = secundaria.reverse()
print("Diagonal principal: ", principal)
print("Diagonal secundária: ", sec)

```

3. O programa abaixo preenche uma matriz 4 x 4 com números inteiros variando entre 0 e 9, inclusive:

```

import random
mat = []
for i in range(4):
    nova_linha = []
    for j in range(4):
        nova_linha.append(random.randint(0,9))
    mat.append(nova_linha)

```

Complete o código, para que ofereça ao usuário um menu com as opções a seguir e execute a opção desejada:

- [1] Imprime a matriz: apresenta a matriz, linha a linha;
- [2] Altera valor: altera o valor da posição (linha e coluna) escolhida pelo usuário;
- [3] imprime posições: imprime as posições (linha e coluna) em que se encontra um número escolhido pelo usuário ou a informação que ele não foi encontrado na matriz;
- [4] imprime linha: imprime uma linha escolhida pelo usuário, que deve informar o número da linha;
- [5] imprime a diagonal principal;
- [6] Encerrar.

```

import random
mat = []
for i in range(4):
    nova_linha = []
    for j in range(4):
        nova_linha.append(random.randint(0,9))
    mat.append(nova_linha)
    print(mat[i])
print("Opções:\n"+
      "[1] - imprime a matriz\n"+
      "[2] - altera valor\n"+
      "[3] - imprime posições\n"+
      "[4] - imprime linha\n"+
      "[5] - imprime a diagonal principal\n"+
      "[6] - encerrar")

opcao = input("Escolha a opção [1 a 6]: ")

while opcao != "6":
    if opcao == "1":
        for i in range(len(mat)):
            print(mat[i])
    elif opcao == "2":
        i = int(input("Linha do valor que deseja alterar: "))
        j = int(input("Coluna do valor que deseja alterar: "))
        mat[i][j] = int(input("Novo valor para essa posição: "))
    elif opcao == "3":
        num = int(input("Valor que deseja saber a posição: "))
        encontrado = False
        for i in range(4):
            for j in range(4):
                if mat[i][j] == num:
                    print(f"Posição de {num}: linha {i}; coluna {j}")
                    encontrado = True
        if not encontrado:
            print("Não tem esse número na matriz")
    elif opcao == "4":
        linha = int(input("Qual linha quer imprimir? "))
        print(mat[linha])
    elif opcao == "5":
        principal = []
        for i in range(4):
            principal.append(mat[i][i])
        print(principal)
    opcao = input("Escolha a opção [1 a 6]: ")

```

## 4.4 Exercícios propostos 3

1. Crie um programa que gere uma matriz 4x4 com números aleatórios entre 1 e 10, imprima essa matriz gerada e então:
  - Calcule e exiba a soma de todos os elementos dessa matriz;
  - Calcule a média dos elementos da matriz;
  - Conte quantos números pares e ímpares existem na matriz.

2. Crie um programa que pergunte ao usuário quantas linhas e quantas colunas terá uma matriz, cujos elementos serão números inteiros aleatórios entre 1 e 10 e permita ao usuário alterar um valor específico dessa matriz, informando a posição a ser alterada e o novo valor. Após a alteração, a matriz atualizada deverá ser exibida.
3. Crie um programa que gere uma matriz 4x4 com números aleatórios entre 1 e 10, imprima essa matriz e crie uma nova matriz transposta a essa. (A matriz transposta de uma matriz A é obtida trocando suas linhas por colunas)
4. Crie um programa que gere uma matriz 3x3 com números aleatórios entre 1 e 9, imprima essa matriz e em seguida troque todos os elementos pares dessa matriz por 0, e imprima novamente a matriz.



## 5 Métodos em listas

### 5.1 Códigos para testes

Códigos 5.1A e 5.1B:

```
lista = [[1,2,3],[4,5,6],[7,8,9]]  
  
for numeros in lista:  
    print(numeros)
```

```
lista = [[1,2,3],[4,5,6],[7,8,9]]  
  
for i in range(len(lista)):  
    print(lista[i])
```

Impressão / Saída:

```
[1, 2, 3]  
[4, 5, 6]  
[7, 8, 9]
```

Código 5.2A e 5.2B:

```
lista = [[1,2,3],[4,5,6],[7,8,9]]  
  
for numeros in lista:  
    print(numeros[1])
```

```
lista = [[1,2,3],[4,5,6],[7,8,9]]  
  
for i in range(len(lista)):  
    print(lista[i][1])
```

Impressão / Saída:

```
2  
5  
8
```

Código 5.3:

```
lista1 = ['A','B','C','D']  
lista2 = ['X','Y','Z']  
lista3 = ['M','N']  
lista4 = ['P','Q','R']  
  
print('Lista 1:', lista1)  
print('Lista 2:', lista2)  
print('Lista 3:', lista3)  
print('Lista 4:', lista4)
```

Impressão / Saída:

```
Lista 1: ['A', 'B', 'C', 'D']  
Lista 2: ['X', 'Y', 'Z']  
Lista 3: ['M', 'N']  
Lista 4: ['P', 'Q', 'R']
```

Código 5.4:

```
lista2 = ['X','Y','Z']  
lista3 = ['M','N']  
  
lista3.extend(lista2)  
print('Lista 3:', lista3)  
lista3.append(lista2)  
print('Lista 3:', lista3)
```

Impressão / Saída:



```
Lista 3: ['M', 'N', 'X', 'Y', 'Z']
Lista 3: ['M', 'N', 'X', 'Y', 'Z', ['X', 'Y', 'Z']]
```

Código 5.5:

```
lista2 = ['X', 'Y', 'Z']
lista4 = ['P', 'Q', 'R']

lista4.append(lista2)
print('Lista 4:', lista4)
lista4.extend(lista2)
print('Lista 4:', lista4)
```

Impressão / Saída:

```
Lista 4: ['P', 'Q', 'R', ['X', 'Y', 'Z']]
Lista 4: ['P', 'Q', 'R', ['X', 'Y', 'Z'], 'X', 'Y', 'Z']
```

Código 5.6:

```
lista1 = ['C', 'B', 'D', 'A']
lista1.sort()
print(lista1)
```

Impressão / Saída:

```
['A', 'B', 'C', 'D']
```

Código 5.7:

```
lista1 = ['A', 'C', 'B', 'D']
print('Lista 1:', lista1)
lista1.insert(3, 'G')
print('Lista 1:', lista1)
lista1.sort()
print('Lista 1:', lista1)
lista1.sort(reverse=True)
print("Lista 1:", lista1)
```

Impressão / Saída:

```
Lista 1: ['A', 'C', 'B', 'D']
Lista 1: ['A', 'C', 'B', 'G', 'D']
Lista 1: ['A', 'B', 'C', 'D', 'G']
Lista 1: ['G', 'D', 'C', 'B', 'A']
```

Código 5.8:

```
lista2 = ['X', 'Y', 'Z']

print('Lista 2:', lista2)
lista2.insert(1, 'Z')
print('Lista 2:', lista2)
lista2.remove('Z')
print('Lista 2:', lista2)
```

Impressão / Saída:

```
Lista 2: ['X', 'Y', 'Z']
Lista 2: ['X', 'Z', 'Y', 'Z']
Lista 2: ['X', 'Y', 'Z']
```

**Código 5.9:**

```
lista3 = ['A', 'B', 'C', 'D', 'M', 'N']

print("lista3.pop() -", lista3.pop())
print('Lista 3:', lista3)
print("lista3.pop(1) -", lista3.pop(1))
print('Lista 3:', lista3)
```

**Impressão / Saída:**

```
lista3.pop() - N
Lista 3: ['A', 'B', 'C', 'D', 'M']
lista3.pop(1) - B
Lista 3: ['A', 'C', 'D', 'M']
```

**Código 5.10:**

```
numeros = [1,2,3,4]
removido = numeros.pop(2)
print(removido)
print(numeros)
```

**Impressão / Saída:**

```
3
[1, 2, 4]
```

**Código 5.11:**

```
lista4 = ['A', 'X', 'C', 'Y', 'M', 'N']
print('Lista 4:', lista4)

lista4[3]='K'
print('Lista 4:', lista4)
lista4.reverse()
print('Lista 4:', lista4)
lista4.sort()
print("Lista4: ", lista4)
lista4.sort(reverse=True)
print("Lista4: ", lista4)
```

**Impressão / Saída:**

```
Lista 4: ['A', 'X', 'C', 'Y', 'M', 'N']
Lista 4: ['A', 'X', 'C', 'K', 'M', 'N']
Lista 4: ['N', 'M', 'K', 'C', 'X', 'A']
Lista4: ['A', 'C', 'K', 'M', 'N', 'X']
Lista4: ['X', 'N', 'M', 'K', 'C', 'A']
```

**Código 5.12:**

```
lista1 = [1, 2, 3]
lista2 = [4, 5, 6]
lista3 = lista1 + lista2
print(lista3)
```

**Impressão / Saída:**

```
[1, 2, 3, 4, 5, 6]
```

#### Código 5.13:

```
lista = [1,2,3,4,5,6,7,8,9]
print("Fatiamento[1:4]- ", lista[1:4])

print("Fatiamento[:3]- ", lista[:3])

print("Fatiamento[3:]- ", lista[3:])

print("Elemento[-1]- ", lista[-1])

print("Elemento[-3]- ", lista[-3])

print("Fatiamento[:]- ", lista[:])
```

#### Impressão / Saída:

```
Fatiamento[1:4]-  [2, 3, 4]
Fatiamento[:3]-  [1, 2, 3]
Fatiamento[3:]-  [4, 5, 6, 7, 8, 9]
Elemento[-1]-  9
Elemento[-3]-  7
Fatiamento[:]-  [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

#### Código 5.14:

```
lista = [1,2,3,4,5,6,7,8,9]

lista[1:3] = [22,33]
print(lista)

lista[-1] = 10
print(lista)
```

#### Impressão / Saída:

```
[1, 22, 33, 4, 5, 6, 7, 8, 9]
[1, 22, 33, 4, 5, 6, 7, 8, 10]
```

## 5.2 Exercícios propostos 4

1. Crie uma lista aleatória de 5 números inteiros e imprima essa lista. Em seguida, adicione 3 outros números digitados pelo usuário nas posições indicadas por ele. Ordene essa lista em ordem decrescente e imprima novamente essa lista.
2. Seja a seguinte lista de frutas: ["manga", "amora", "laranja", "banana", "melancia"]. Solicite ao usuário que digite o nome de uma fruta e avise se ela está ou não nessa lista.
3. Construa com valores aleatórios uma lista com 10 números inteiros entre 1 e 10; imprima essa lista e depois crie e imprima uma nova lista contendo apenas os números ímpares dessa lista.

## 6 Manipulação de strings

### 6.1 Métodos de manipulação de strings

- a) `.upper()`: Converte toda a string para **maiúsculas**.
- b) `.lower()`: Converte toda a string para **minúsculas**.
- c) `.title()`: Converte **cada palavra** para **começar com maiúscula**.
- d) `.capitalize()`: Converte apenas o **primeiro caractere** para maiúscula.

### 6.2 Códigos para testes

Código 6.1:

```
texto = "python é incrível!"

print(texto.upper())
print(texto.lower())
print(texto.title())
print(texto.capitalize())
```

Impressão / Saída:

```
PYTHON É INCRÍVEL!
python é incrível!
Python É Incrível!
Python é incrível!
```

Código 6.2:

```
op = input("Opção:")

if op.lower() == "sim":
    print("sim")
else:
    print("nao")
```

Impressão / Saída:

```
Opção: Sim
sim
```

Código 6.3:

```
texto = "python é incrível!"

print(texto.upper())
print(texto)
print("****")
print(texto.lower())
print(texto)
print("****")
print(texto.title())
print(texto)
print("****")
print(texto.capitalize())
print(texto)
```

Impressão / Saída:

```
PYTHON É INCRÍVEL!
python é incrível!
***
python é incrível!
python é incrível!
***
Python É Incrível!
python é incrível!
***
Python é incrível!
python é incrível!
```

- e) **.strip()**: Remove espaços em branco no início e no fim da string.
- f) **.lstrip()**: qtas).
- g) **.rstrip()**: Remove espaços apenas no fim (direita).

#### Código 6.4:

```
mensagem = "    Hello, World!    "

print(mensagem.strip())
print(mensagem.lstrip())
print(mensagem.rstrip())
```

#### Impressão / Saída:

```
Hello, World!
Hello, World!
    Hello, World!
```

- h) **.replace(antigo, novo)**: Substitui uma parte da string por outra.
- i) **.find(valor)**: Retorna o índice da primeira ocorrência do valor ou -1 se não encontrar.
- j) **.count(valor)**: Conta quantas vezes o valor aparece na string.

#### Código 6.5:

```
frase = "O céu é azul e o mar também é azul."

print(frase.replace("azul", "verde"))
print(frase.find("céu"))
print(frase.count("azul"))
```

#### Impressão / Saída:

```
O céu é verde e o mar também é verde.
2
2
```

- k) **.split(separador)**: Divide a string em uma lista de substrings, usando um separador (padrão: espaço).
- l) **.join(lista)**: Junta os elementos de uma lista em uma string, colocando um separador entre eles.

#### Código 6.6:

```
dados = "Maçã, Banana, Uva, Laranja"

lista_frutas = dados.split(", ")
print(lista_frutas)

nova_frase = " - ".join(lista_frutas)
print(nova_frase)
```

#### Impressão / Saída:

```
['Maçã', 'Banana', 'Uva', 'Laranja']
Maçã - Banana - Uva - Laranja
```

- m) **.isnumeric()**: retorna *True* se a string contiver apenas números e *False* caso contrário.



n) **.isalpha()**: retorna *True* se a string contiver apenas letras e *False* caso tenha números, espaços ou símbolos.

Código 6.7:

```
print("Ex.1 - '12345':")
print("12345".isnumeric())
print("****")
print("Ex.2 - '123 456':")
print("123 456".isnumeric())
print("****")
print("Ex.3 - '12.34':")
print("12.34".isnumeric())
print("****")
print("Ex.4 - '12,34':")
print("12,34".isnumeric())
print("****")
print("Ex.5 - 'mil':")
print("mil".isnumeric())
```

Impressão / Saída:

```
Ex.1 - '12345':
True
***
Ex.2 - '123 456':
False
***
Ex.3 - '12.34':
False
***
Ex.4 - '12,34':
False
***
Ex.5 - 'mil':
False
```

Código 6.8:

```
print("Ex.1 - 'abcde':")
print("abcde".isalpha())
print("****")
print("Ex.2 - 'áéíóúÃÊÍÓÚ':")
print("áéíóúÃÊÍÓÚ".isalpha())
print("****")
print("Ex.3 - 'Hello!':")
print("Hello!".isalpha())
print("****")
print("Ex.4 - 'Hello, World!':")
print("Hello, World!".isalpha())
print("****")
```

Impressão / Saída:

```

Ex.1 - 'abcde':
True
***
Ex.2 - 'áéíóúÃĖİÓŮ':
True
***
Ex.3 - 'Hello!':
False
***
Ex.4 - 'Hello, World!':
False
***

```

### 6.3 Fatiamiento de strings

O fatiamiento de strings permite extrair partes específicas de uma string usando a sintaxe:

`string[início:fim:passo]`

- **início** → Índice onde a fatia começa (inclusivo).
- **fim** → Índice onde a fatia termina (**exclusivo**).
- **passo** → Determina de quantos em quantos caracteres a extração ocorre.

Se **início** for omitido, assume o início da string (0).

Se **fim** for omitido, assume o final da string.

Se **passo** for negativo, a string será lida de **trás para frente**.

Código 6.7:

```

texto = "Python"

print(texto[0:4])
print(texto[:3])
print(texto[3:])
print(texto[-3:])
print(texto[::-1])
print(texto[::2])

```

Impressão / Saída:

```

Pyth
Pyt
hon
hon
nohtyP
Pto

```

### 6.4 Exercícios resolvidos

1. Escreva um código que solicite um e-mail e depois imprima a primeira parte do e-mail e o seu servidor.

```

email = input("Digite o seu e-mail: ")
pos = email.find("@")
print("início do e-mail: ", email[:pos])
print("servidor: ", email[pos+1:])

```



2. Solicite o nome completo do usuário e depois imprime somente o primeiro nome

```
nome_completo = input("Digite seu nome completo: ")
primeiro_nome = nome_completo.split()[0]
print("Primeiro nome:", primeiro_nome)
```

3. Escreva um código que receba uma string do usuário e exiba: a) o primeiro caractere; b) o último caractere; c) os três primeiros caracteres; d) os três últimos caracteres.

```
texto = input("Digite uma palavra: ")
print("Primeiro caractere:", texto[0])
print("Último caractere:", texto[-1])
print("Três primeiros:", texto[:3])
print("Três últimos:", texto[-3:])
```

4. Receba uma string e exiba a string ao contrário.

```
texto = input("Digite uma palavra ou frase: ")
texto_invertido = texto[::-1]
print("String invertida:", texto_invertido)
```

```
texto = input("Digite uma palavra ou frase: ")
texto_invertido = ""
for caractere in texto:
    texto_invertido = caractere + texto_invertido
```

5. Digite uma palavra e imprima a metade dela.

```
palavra = input("Digite uma palavra: ")
meio = len(palavra)//2
print("Metade inicial:", palavra[:meio])
```

## 6.5 Exercícios propostos 5

1. Peça ao usuário uma frase e em seguida imprima essa frase sem as vogais.
2. Crie um programa para cadastro de CPF de clientes que recebe o CPF do usuário apenas com números. Caso o usuário digite algo diferente de números ou digite menos de 11 caracteres (tamanho do CPF brasileiro), o programa deve exibir uma mensagem de "Digite seu CPF corretamente e digite apenas números"
3. Agora, além das validações anteriores, permita que o usuário insira pontos, traços e inclusive espaços vazios. E o script deve "tratar" o que o usuário inserir para padronizar o CPF dele em apenas números. retirar todos os pontos, traços e espaços do CPF que o cliente inserir e, uma vez retirados pontos, traços e espaços, devem sobrar apenas números no CPF. Qualquer outro caractere deve ser considerado inválido.
4. Seja dada uma lista de clientes devedores com as informações: CPF, valor devido e dias de atraso. Deverá ser criada uma lista com o CPF dos clientes devedores que a empresa considera como

inadimplentes, sendo que será considerado inadimplente o cliente que tiver devendo mais que R\$ 2000,00 por mais de 20 dias. Informe qual o percentual de clientes considerados inadimplentes em relação a quantidade de clientes devedores.

```
clientes_devedores=[['456.810.836-41', 3192.50, 18], ['783.613.218-67', 3447.43, 25],  
                    ['989.379.232-65', 2584.36, 34], ['748.663.779-88', 2487.45, 38],  
                    ['264.760.944-20', 1960.34, 28]]
```

5. Peça ao usuário que insira o seu nome completo. Faça a validação dessa entrada verificando se há somente letras (caso tenham sido caracteres que não seja letras, informe ao usuário que o nome deve conter apenas letras), remova espaços antes e depois do nome e garanta que todas as palavras estejam começando com letras maiúsculas, e então imprima o nome completo formatado.
6. Peça uma frase e conte quantas letras e quantos números há na frase usando os métodos `isnumeric()` e `isalpha()`.
7. Peça ao usuário uma placa de um veículo e verifique se essa placa é válida e a qual modelo ela pertence: modelo antigo (ex. ABC-1234) ou modelo Mercosul (ABC1D23)

## 7 Functions

Funções são blocos de código reutilizáveis que realizam uma tarefa específica.

Tornam os programas mais organizados e fáceis de manter, evitando repetição de código.

### 7.1 Estrutura básica

```
def nome_funcao():  
    faça alguma coisa  
    faça outra coisa  
    return valor_final
```

Pode-se implementar funções sem parâmetros e/ou sem o return.

Pode-se usar no return praticamente qualquer tipo de objeto: (número, string, lista, tupla, dicionário, outros objetos, etc.)

O return, se for executado, encerra a função, mesmo que dentro dela haja um loop.

### 7.2 Exemplos de funções

Ex. 1 - Código inicial sem uso de funções:

```
print ("Olá, João! Tudo bem com você? ")  
print ("Olá, Maria! Tudo bem com você? ")  
print ("Olá, Ana! Tudo bem com você? ")
```

Impressão / Saída:

```
Olá, João! Tudo bem com você?  
Olá, Maria! Tudo bem com você?  
Olá, Ana! Tudo bem com você?
```

Ex. 2 - Código reescrito com uso de funções:

```
def fazer_saudacao(nome):  
    print (f"Olá, {nome}! Tudo bem com você? ")  
  
fazer_saudacao("João")  
fazer_saudacao("Maria")  
fazer_saudacao("Ana")  
  
def fazer_saudacao(nome):  
    print (f"Olá, {nome}! Tudo bem com você? ")  
  
lista = ["João", "Maria", "Ana"]  
for pessoa in lista:  
    fazer_saudacao(pessoa)
```

Impressão / Saída:

```
Olá, João! Tudo bem com você?  
Olá, Maria! Tudo bem com você?  
Olá, Ana! Tudo bem com você?
```

Ex. 3 - Código sem parâmetros e sem retorno:

```
def mensagem():  
    print("Começando os estudos de funções em Python...")  
  
mensagem()
```

Impressão / Saída:

```
Começando os estudos de funções em Python ...
```

**Ex. 4 - Código com parâmetros e sem retorno:**

```
def dobro (num) :  
    print(f"O dobro de {num} é {num * 2}")  
  
dobro(5)  
dobro(10)
```

**Impressão / Saída:**

```
O dobro de num é: 10  
O dobro de num é: 20
```

**Ex. 5 - Código com parâmetros e sem retorno (*None* como saída de impressão)**

```
def dobro (num) :  
    print(f"O dobro de num é: {num*2}")  
  
print(dobro (5))  
print(dobro (10))
```

**Impressão / Saída:**

```
O dobro de num é: 10  
None  
O dobro de num é: 20  
None
```

**Ex. 6 - Código com parâmetros e com retorno:**

```
def soma(a, b) :  
    return a + b  
  
resultado = soma(4, 7)  
print(f"A soma é {resultado}")
```

**Impressão / Saída:**

```
A soma é 11
```

**Ex. 7 - Código com parâmetros e com retorno:**

```
def soma(a, b) :  
    if a > b:  
        return a + b  
  
resultado = soma(4,7)  
print(f"A soma é {resultado}")
```

**Impressão / Saída:**

```
A soma é None
```

**Ex. 8 – Função que retorna um número:**

```
def minha_funcao(parametro1, parametro2, parametro3) :  
    return (parametro1 + parametro2 + parametro3)  
  
print(minha_funcao(1,2,3))
```

**Impressão / Saída:**

```
6
```

Ex. 9 – Função que retorna um número:

```
def minha_funcao(parametro1, parametro2, parametro3):  
    return (parametro1 + parametro2 + parametro3)  
  
minha_funcao(10,20,30)  
print(minha_funcao(1,2,3))
```

Impressão / Saída:

6

Ex. 10 – Função que retorna um número:

```
def minha_funcao(parametro1, parametro2, parametro3):  
    return (parametro1 + parametro2 + parametro3)  
  
result = minha_funcao(10,20,30)  
print (result)  
print(minha_funcao(1,2,3))
```

Impressão / Saída:

60

6

Ex. 11 – Função que retorna um texto:

```
def padronizar_texto(texto):  
    texto = texto.upper()  
    texto = texto.replace ("!", "")  
    texto = texto.strip()  
    return (texto)  
  
mensagem = "    Olá, todo mundo!  "  
print(padronizar_texto(mensagem))
```

Impressão / Saída:

OLÁ, TODO MUNDO

Ex. 12 – Função que retorna um valor booleano:

```
def bateu_meta(vendas, meta):  
    if vendas >=meta:  
        return True  
    else:  
        return False  
  
if bateu_meta(850, 1000):  
    print("A meta foi alcançada")  
else:  
    print("A meta não foi alcançada")
```

Impressão / Saída:

A meta não foi alcançada

```
def bateu_meta(vendas, meta):
    if vendas > meta:
        return True

vendas = [850, 1300, 1500, 700, 2000]

for valor_venda in vendas:
    if bateu_meta(valor_venda, 1000):
        print(f"O valor {valor_venda} bateu a meta")
    else:
        print(f"O valor {valor_venda} não bateu a meta")
```

Impressão / Saída:

```
O valor 850 não bateu a meta
O valor 1300 bateu a meta
O valor 1500 bateu a meta
O valor 700 não bateu a meta
O valor 2000 bateu a meta
```

Ex. 13 – Função que retorna uma lista:

```
def filtrar_lista_texto (lista, pedaco_texto):
    lista_filtrada = []
    for item in lista:
        if pedaco_texto in item:
            lista_filtrada.append(item)
    return lista_filtrada

print(filtrar_lista_texto(("AB123", "RT234", "CE456", "AB345"), "AB"))
```

Impressão / Saída:

```
['AB123', 'AB345']
```

## 7.3 Funções embutidas vs. Funções do usuário

O Python já possui várias **funções embutidas** (built-in), como:

**input(mensagem)** → Captura entrada do usuário como string.

**print(objeto,...)** → Exibe algo na tela

**len(seq)** → Retorna o tamanho de uma sequência

**sum(iterável)** → Soma todos os elementos de uma sequência

**max(iterável)** → Retorna o maior valor de uma sequência

**min iterável ()** → Retorna o menor valor de uma sequência

**int(valor)** → Converte para inteiro.

**float(valor)** → Converte para ponto flutuante.

**str(valor)** → Converte para string.

**bool(valor)** → Converte para booleano (True ou False).

**list(iterável)** → Converte um iterável para lista.

**tuple(iterável)** → Converte um iterável para tupla.



Código 7.2: Função que recebe três notas e retorna a média do aluno

```
def calcular_media(n1, n2, n3):  
    media = (n1 + n2 + n3) / 3  
    return media  
  
resultado = calcular_media(8, 7, 9)  
print(f"A média do aluno é {resultado}")
```

Impressão / Saída:

A média do aluno é 8.0

Código 7.3: Função de cadastro de um produto. Essa função garante que o produto cadastrado está em letra minúscula

```
def cadastrar_produto():  
    produto = input("Digite o nome do produto que deseja cadastrar: ")  
    produto = produto.casefold()  
    print("Produto {} cadastrado com sucesso".format(produto))  
  
for i in range(3):  
    cadastrar_produto()
```

Impressão / Saída:

Digite o nome do produto que deseja cadastrar: feijAO  
Produto feijao cadastrado com sucesso  
Digite o nome do produto que deseja cadastrar: Arroz  
Produto arroz cadastrado com sucesso  
Digite o nome do produto que deseja cadastrar: MACARRAO  
Produto macarrao cadastrado com sucesso

## 7.5 Exercícios resolvidos - 1

1. Crie um código que execute uma função que recebe três notas e retorna a média do aluno.

```
def calcular_media(n1, n2, n3):  
    media = (n1 + n2 + n3) / 3  
    return media  
  
resultado = calcular_media(8, 7, 9)  
print(f"A média do aluno é {resultado}")
```

2. Crie uma função chamada eh\_par() que recebe um número e retorna True se o número for par, ou False se o número for ímpar.

```
def eh_par(numero):  
    return numero%2 == 0
```

3. Crie um código que use essa função eh\_par criada no exercício 1, solicite um número ao usuário e informe se ele é par ou ímpar.



```
def eh_par(numero):
    return numero%2 == 0

num = int(input("Digite um número: "))
if eh_par(num):
    print(f"O número {num} é par")
else:
    print(f"O número {num} é ímpar")
```

4. Crie uma função chamada `contar_vogais` que recebe uma palavra e retorna a quantidade de vogais nessa palavra.

```
def contar_vogais(palavra):
    vogais = "aeiouAEIOU"
    contador = 0
    for letra in palavra:
        if letra in vogais:
            contador += 1
    return contador
```

## 7.6 Argumentos das funções

Quando chamamos uma função e passamos valores para ela, esses valores podem ser passados de duas formas:

- Em ordem (argumentos posicionais)
- Com o nome do argumento (keyword argument)

**Código 7.4: Uso de argumentos posicionais:**

```
def apresentar(nome, idade):
    print(f"Meu nome é {nome} e tenho {idade} anos.")

apresentar("Alice", 25)
apresentar("Carlos", 30)
```

**Impressão / Saída:**

```
Meu nome é Alice e tenho 25 anos.
Meu nome é Carlos e tenho 30 anos.
```

- No caso de argumentos posicionais, se a ordem dos parâmetros for invertida, o resultado muda ou pode causar erros.

```
def apresentar(nome, idade):
    print(f"Meu nome é {nome} e tenho {idade} anos.")

apresentar(25, "Alice")
apresentar(30, "Carlos")
```

**Impressão / Saída:**

```
Meu nome é 25 e tenho Alice anos.
Meu nome é 30 e tenho Carlos anos.
```

**Código 7.4: Uso de argumentos nomeados:**

```
def apresentar(nome, idade):
    print(f"Meu nome é {nome} e tenho {idade} anos.")

apresentar(nome="Alice", idade=25)
apresentar(idade=30, nome="Carlos")
```

Impressão / Saída:

```
Meu nome é Alice e tenho 25 anos.
Meu nome é Carlos e tenho 30 anos.
```

**Código 7.5: Argumentos com valores padrão (Default arguments):**

```
def apresentar(nome, idade=18):
    print(f"Meu nome é {nome} e tenho {idade} anos.")

apresentar("Alice")
apresentar("Carlos", 30)
```

Impressão / Saída:

```
Meu nome é Alice e tenho 18 anos.
Meu nome é Carlos e tenho 30 anos.
```

*Obs: Argumentos com valores padrão devem vir depois dos obrigatórios*

**Código 7.6: Argumentos posicionais variáveis (\*args).**

**\*args** permite que a função receba vários argumentos posicionais como uma tupla. Pode-se iterar sobre args ou usar funções como `sum()` para processar os valores.

```
def somar(*numeros):
    total = sum(numeros)
    print(f"Soma total: {total}")

somar(1, 2, 3)
somar(10, 20, 30, 40, 50)
```

Impressão / Saída:

```
Soma total: 6
Soma total: 150
```

**Código 7.6: Argumentos posicionais variáveis (\*args).**

```
def listar_nomes(*nomes):
    print("Lista de nomes recebidos:")
    for nome in nomes:
        print(f"- {nome}")

listar_nomes("Alice", "Bruno", "Carlos")
listar_nomes("Mariana", "João")
```

Impressão / Saída:

```
Lista de nomes recebidos:
- Alice
- Bruno
- Carlos
Lista de nomes recebidos:
- Mariana
- João
```

## 7.7 Exercícios resolvidos - 2

1. Crie uma função que faça uma contagem\_regressiva (n) que imprima uma contagem de n até 0.

Usando loop com while ou loop com for:

```
def contagem_regressiva(n):  
    while n >= 0:  
        print (n)  
        n -= 1
```

```
def contagem_regressiva(n):  
    for i in range (n,-1,-1):  
        print (i)
```

2. Implemente uma função maior(a, b), que retorne o maior número entre a e b.

```
def maior(a,b):  
    if a > b:  
        return a  
    else:  
        return b
```

3. Escreva uma função que receba dois números e retorne True se um dos números for 10 ou se a soma dos dois números for 10.

4. Crie uma função **fatorial(n)** que retorne o fatorial de n (n!). O fatorial de um número inteiro positivo n é o produto de todos os números inteiros positivos de 1 até n.

```
def fatorial(n):  
    resultado = 1  
    for i in range(1, n + 1):  
        resultado *= i  
    return resultado
```

5. Crie um código que retorne uma lista com os quadrados de 1 a n, usando uma função implementada por você: quadrados (n), no formato:

O quadrado de 1 é 1

O quadrado de 2 é 4

O quadrado de 3 é 9

```
def quadrado (n):  
    for i in range(1, n+1):  
        print(f"O quadrado de {i} é {i**2}")
```

6. Seja a definição de número primo: um número **primo** é um número **maior que 1** que só pode ser dividido por **1 e ele mesmo**. Crie um código que crie uma lista com os números primos até determinado número n e indique quantos são esses números.

```
def eh_primo(num):
    if num < 2:
        return False
    for i in range(2,num):
        if num%i == 0:
            return False
    return True

primos = []
n = int(input("Você quer uma lista de números primos até que número? "))
for i in range(n):
    if eh_primo(i):
        primos.append(i)
print(f"Lista de números primos até {n}: {primos}")
print (f"Há {len(primos)} números primos até o número {n}")
```

## 7.8 Exercícios propostos 6

1. Escreva uma função `media(lista_de_notas)` que receba uma lista de números e retorne a média deles.
2. Escreva uma função `multiplicar(a, b)` que retorne o produto entre dois números.
3. Crie uma função `converter_minutos_em_horas(minutos)` que retorne uma string no formato "X horas e Y minutos".
4. Faça uma função `contar_palavras(frase)` que retorne a quantidade de palavras em uma string.
5. Implemente uma função `eh_bissexto(ano)` que diga se um ano é bissexto ou não. Para saber se um ano é bissexto, devemos verificar se ele se encaixa em um dos casos:
  - a) É um número divisível por 4, mas não é divisível por 100
  - b) É um número divisível por 4, por 100 e por 400
6. Escreva a função `potencia(base, expoente)` que retorna a base elevada ao expoente.
7. Crie uma função `calcular_preco_com_desconto(valor, percentual)` que retorne o valor final com desconto aplicado.
8. Criar um script para categorizar códigos de despesas em uma obra. Cada despesa possui um código inicial "MAT" se a despesa for com materiais (cimento, ferro,..), ou "MOB" se for com mão de obra (pedreiro, servente,..). Há também diversos outros códigos de despesas. O programa deverá separar essa lista original em três outras listas: um dos códigos de despesas com materiais, outra com as despesas com mão de obra e outra com "outras despesas".

```
despesas = ["MOB13", "MAT34", "DIN45", "FIN56", "MOB34", "MAT67", "MAT98"]
```

## 7.9 Respostas dos exercícios propostos 6

Exercício 1:

```
def media(lista):  
    return sum(lista)/len(lista)
```

Exercício 2:

```
def multiplicar(a,b):  
    return a*b
```

Exercício 3:

```
def converter_minutos_em_horas(minutos):  
    horas = minutos//60  
    minutos = minutos%60  
    if horas <= 1:  
        return str(horas) + " hora " + str(minutos) + " minutos"  
    return str(horas) + " horas " + str(minutos) + " minutos"
```

Exercício 4:

```
def contar_palavras(frase):  
    return len(frase.split(" "))
```

Exercício 5:

```
def eh_bissexto(ano):  
    return (ano%4 == 0 and ano%100 != 0) or ano%400 == 0
```

Exercício 6:

```
def potencia(base,expoente):  
    return base**expoente
```

Exercício 7: (considerando que o valor do percentual pode ser digitado em percentual ou em formato decimal):

```
def calcular_preco_com_desconto(valor,percentual):  
    return valor - valor * percentual  
  
print(calcular_preco_com_desconto(1000,0.10))
```

Exercício 7: (considerando que o valor do percentual será digitado em formato decimal):

```
def calcular_preco_com_desconto(valor,percentual):  
    if type(percentual) == str and "%" in percentual:  
        percentual = percentual[:-1]  
        percentual = float(percentual)  
    return valor - valor*percentual/100  
    return valor - valor * percentual  
  
print(calcular_preco_com_desconto(1000,"10%"))  
print(calcular_preco_com_desconto(1000,0.10))
```



Exercício 8 – sem o uso de função:

```
despesas = ["MOB13", "MAT34", "DIN45", "FIN56", "MOB34", "MAT67", "MAT98"]
mao_de_obra = []
materiais = []
outros = []

for codigo in despesas:
    if "MOB" in codigo:
        mao_de_obra.append(codigo)
    elif "MAT" in codigo:
        materiais.append(codigo)
    else:
        outros.append(codigo)

print("Lista de mão de obra: ",mao_de_obra)
print("Lista de materiais: ",materiais)
print("Lista de outras despesas: ",outros)
```

Exercício 8 – com o uso de função:

```
def testar_codigo(desp, cod):
    if cod in desp:
        return True

despesas = ["MOB13", "MAT34", "DIN45", "FIN56", "MOB34", "MAT67", "MAT98"]
mao_de_obra = []
materiais = []
outros = []

for despesa in despesas:
    if testar_codigo(despesa, "MOB"):
        mao_de_obra.append(despesa)
    elif testar_codigo(despesa, "MAT"):
        materiais.append(despesa)
    else:
        outros.append(despesa)

print("Lista de mão de obra: ",mao_de_obra)
print("Lista de materiais: ",materiais)
print("Lista de outras despesas: ",outros)
```

## 7.10 Passagem por valor e por referência

O Python mantém uma tabela com os nomes das variáveis e para quais objetos “apontam”.

Há variáveis imutáveis: objetos associados às variáveis do tipo int, float, string, bool e tuplas.

Obs. As Tuplas são estruturas de dados utilizadas em Python que funcionam de modo semelhante a uma lista, porém são imutáveis.

Código 7.7:

```
a = [1,5]
b = [1,5]
print (a,b)

a[0]=9
b.append(3)
print(a,b)
```

Impressão / Saída:

```
[1, 5] [1, 5]
[9, 5] [1, 5, 3]
```

Nesse código, **a** e **b** são duas listas diferentes, apesar de terem o mesmo conteúdo inicialmente. [1,5] é usado duas vezes, criando dois objetos distintos na memória.

Código 7.8:

```
a = b = [1,5]
print (a,b)

a[0]=9
b.append(3)
print(a,b)
```

Impressão / Saída:

```
[1, 5] [1, 5]
[9, 5, 3] [9, 5, 3]
```

Já nesse código, **a** e **b** apontam para a mesma lista! Ou seja, são duas variáveis que compartilham o mesmo objeto na memória.

Código 7.9: (passagem por valor)

```
def valor(x):
    x = 1
    print("x dentro da função: ", x)

x = 0
print("x antes da função: ", x)
valor(x)
print("x depois da função: ", x)
```

Impressão / Saída:

```
x antes da função: 0
x dentro da função: 1
x depois da função: 0
```

Quando se usa em uma função um argumento imutável, faz-se uma passagem por valor. Qualquer modificação na variável dentro da função não altera (não muda) seu valor fora da mesma.



Código 7.10: (passagem por referência)

```
def referencia(y):  
    y[0] = 1  
    y.append(5)  
    print("y dentro da função: ", y)  
  
y = [0,0]  
print("y antes da função: ", y)  
referencia(y)  
print("y depois da função: ", y)
```

Impressão / Saída:

```
y antes da função:  [0, 0]  
y dentro da função:  [1, 0, 5]  
y depois da função:  [1, 0, 5]
```

Código 7.10: (passagens por referência e por valor)

```
def modificar(a, b, c, d, e):  
    a = 22  
    b = (33, 44)  
    c.append(42)  
    d[0] = 55  
    e = "Olá"  
    print("Dentro da função: ")  
    print("a:", a, "b:", b, "c:", c, "d:", d, "e:", e)  
  
a = 77  
b = (15, 10)  
c = [88, 99]  
d = [28, 23]  
e = "oi"  
  
print("Antes da função: ")  
print("a:", a, "b:", b, "c:", c, "d:", d, "e:", e)  
modificar(a, b, c, d, e)  
print("Depois da função: ")  
print("a:", a, "b:", b, "c:", c, "d:", d, "e:", e)
```

Impressão / Saída:

```
Antes da função:  
a: 77 b: (15, 10) c: [88, 99] d: [28, 23] e: oi  
Dentro da função:  
a: 22 b: (33, 44) c: [88, 99, 42] d: [55, 23] e: Olá  
Depois da função:  
a: 77 b: (15, 10) c: [88, 99, 42] d: [55, 23] e: oi
```

## 7.11 Testes de mesa – códigos com funções

### Teste 7.1:

```
def somar(a, b):  
    return a + b  
  
print(somar(10, somar(20, somar(30, somar(40, 50)))))
```

Impressão / Saída:

150

### Teste 7.2:

```
def funcao1(x):  
    a = 0  
    for i in x:  
        if i%2 == 0:  
            a = a + i  
    return a  
  
lista = [1,2,3,4,5,6,7,8,9]  
print(funcao1(lista))
```

Impressão / Saída:

20

### Teste 7.3:

```
def produto(x):  
    return x * x  
  
def funcao2(x,y,z):  
    a = produto(x)  
    b = produto(y)  
    c = produto(z)  
    print(x,y,z)  
    return a+b+c  
  
a = -5  
b = 2  
c = 10  
funcao2(a,b,c)
```

Impressão / Saída:

-5 2 10

#### Teste 7.4:

```
def produto(x):  
    return x * x  
  
def funcao2(x,y,z):  
    a = produto(x)  
    b = produto(y)  
    c = produto(z)  
    print (x,y,z)  
    return a+b+c  
  
a = -5  
b = 2  
c = 10  
print(funcao2(a,b,c))
```

Impressão / Saída:

-5 2 10  
129

#### Teste 7.5:

```
def produto(x):  
    return x * x  
  
def funcao2(x,y,z):  
    a = produto(x)  
    b = produto(y)  
    c = produto(z)  
    return a+b+c  
  
a = -5  
b = 2  
c = 10  
print(funcao2(a,b,c))
```

Impressão / Saída:

129

## 8 Dicionários em Python

### 8.1 Conceitos iniciais

A estrutura dos dicionários é: {chave: valor, chave: valor, chave: valor}

Os dicionários são mais intuitivos de trabalhar;

Não devem ser usados para pegar itens em uma determinada ordem;

As chaves são únicas obrigatoriamente;

Podem ter valores heterogêneos (inteiros, strings, listas)

O acesso aos valores é feito usando-se a chave do dicionário:

valor = dicionario[chave]

#### Exemplo 8.1:

```
vendas_loja1 = {"calças":24, "shorts": 35, "camisetas": 42, "bonés": 54}
print (vendas_loja1["calças"])
print (vendas_loja1["bonés"])
```

Impressão / Saída:

24

54

### 8.2 Acessando valores nos dicionários

O valor pode ser acessado pelo método .get(chave). Nesse caso, não existindo a chave no dicionário, o método retorna None.

#### Exemplo 8.2:

```
vendas = {"calças":24, "shorts": 35, "camisetas": 42, "bonés": 54}
print(vendas.get("shorts"))
print(vendas.get("bonés"))
print(vendas.get("vestidos"))
```

Impressão / Saída:

35

54

None

#### Exemplo 8.3:

```
vendas = {"calças":24, "shorts": 35, "camisetas": 42, "bonés": 54}
if "vestidos" in vendas:
    print (vendas["vestidos"])
else:
    print ("vestidos não está no dicionário")
```

Impressão / Saída:

vestidos não está no dicionário

#### Exemplo 8.4:

```
vendas = {"calças":24, "shorts": 35, "camisetas": 42, "bonés": 54}
if "calças" in vendas:
    print (vendas["calças"])
else:
    print ("calças não está no dicionário")
```

Impressão / Saída:

24

### 8.3 Alterando valores no dicionário

#### Exemplo 8.5:

```
vendas = {"calças":24, "shorts": 35, "camisetas": 42, "bonés": 54}

vendas["vestidos"] = 50

print (vendas)
```

Impressão / Saída:

{'calças': 24, 'shorts': 35, 'camisetas': 42, 'bonés': 54, 'vestidos': 50}

#### Exemplo 8.6:

```
vendas = {"calças":24, "shorts": 35, "camisetas": 42, "bonés": 54}
print (vendas)

vendas_camisetas = vendas["camisetas"]
print (vendas_camisetas)

vendas["camisetas"] = 56
print (vendas)
vendas_camisetas = vendas["camisetas"]

print (vendas_camisetas)
```

Impressão / Saída:

{'calças': 24, 'shorts': 35, 'camisetas': 42, 'bonés': 54}  
42  
{'calças': 24, 'shorts': 35, 'camisetas': 56, 'bonés': 54}  
56

#### Exemplo 8.7:

```
temp_1tri = {"jan": 30, "fev": 26, "mar": 35}
print (temp_1tri)
temp_1tri["abr"] = 34
print (temp_1tri)
temp_1tri.update({"mai": 29, "jun": 32})
print (temp_1tri)
```

Impressão / Saída:

{'jan': 30, 'fev': 26, 'mar': 35}  
{'jan': 30, 'fev': 26, 'mar': 35, 'abr': 34}  
{'jan': 30, 'fev': 26, 'mar': 35, 'abr': 34, 'mai': 29, 'jun': 32}



**Exemplo 8.8:**

```
temp_1tri = {"jan": 30, "fev": 26, "mar": 35, "abr": 34, "mai": 29, "jun": 32}
temp_2tri = {"jul": 35, "ago": 40, "set": 39, "out": 42, "nov": 33, "dez": 29}
temp_1tri.update(temp_2tri)
print (temp_1tri)
```

Impressão / Saída:

```
{'jan': 30, 'fev': 26, 'mar': 35, 'abr': 34, 'mai': 29, 'jun': 32,
'jul': 35, 'ago': 40, 'set': 39, 'out': 42, 'nov': 33, 'dez': 29}
```

**Exemplo 8.9:** Chaves são únicas

```
temp_1tri = {"jan": 30, "fev": 26, "mar": 35, "abr": 34, "mai": 29, "jun": 32}
temp_1tri["jan"]=29
print (temp_1tri)
```

Impressão / Saída:

```
{'jan': 29, 'fev': 26, 'mar': 35, 'abr': 34, 'mai': 29, 'jun': 32}
```

**Exemplo 8.10:** Tentando criar um dicionário com duas chaves iguais

```
temp_1tri = {"jan": 30, "fev": 26, "mar": 35, "jan": 29}
print (temp_1tri)
```

Impressão / Saída:

```
{'jan': 29, 'fev': 26, 'mar': 35}
```

**Exemplo 8.11:** Retirando item do dicionário

```
temp_1tri = {"jan": 30, "fev": 26, "mar": 35}

del temp_1tri["jan"]

print (temp_1tri)
```

Impressão / Saída:

```
{'fev': 26, 'mar': 35}
```

**Exemplo 8.12:** Retirando item do dicionário com .pop()

```
temp_1tri = {"jan": 30, "fev": 26, "mar": 35}

temp_jan = temp_1tri.pop("jan")
print (f"Temperatura em janeiro: {temp_jan}")
print (temp_1tri)
```

Impressão / Saída:

```
Temperatura em janeiro: 30
{'fev': 26, 'mar': 35}
```

**Exemplo 8.13:** Apagando todos os dados do dicionário sem deletar o dicionário, usando .clear()

```
temp_1tri = {"jan": 30, "fev": 26, "mar": 35}
temp_1tri.clear()
print (temp_1tri)
```

Impressão / Saída:

```
{}
```



## 8.4 Usando o for nos dicionários

```
for chave in dicionario:
    faça alguma coisa
```

### Exemplo 8.14:

```
temp_1tri = {"jan": 30, "fev": 26, "mar": 35}

for chave in temp_1tri:
    print (chave)

for chave in temp_1tri:
    print (f"{chave}: {temp_1tri[chave]}")
```

Impressão / Saída:

```
jan
fev
mar
jan: 30
fev: 26
mar: 35
```

## 8.5 Métodos .items() e .keys()

O método `.keys()` retorna uma view com todas as chaves de um dicionário.

O método `.items()` retorna uma view de pares (**chave, valor**) do dicionário em forma de tuplas.

### Exemplo 8.15:

```
dados = {"Ana": 8.0, "Beto": 7.5}
print(dados.keys())
```

Impressão / Saída:

```
dict_keys(['Ana', 'Beto'])
```

### Exemplo 8.16:

```
dados = {"Ana": 8.0, "Beto": 7.5}
for nome in dados.keys():
    print(nome)
```

Impressão / Saída:

```
Ana
Beto
```

### Exemplo 8.17:

```
dados = {"Ana": 8.0, "Beto": 7.5}
print(dados.items())
```

Impressão / Saída:

```
dict_items([('Ana', 8.0), ('Beto', 7.5)])
```

## 8.6 Exercícios resolvidos

1. Considere o seguinte dicionário de vendas de camisetas e calças P, M e G de uma determinada loja.

```
vendas = {"camP":10,  
          "calçaP":20,  
          "camM":20,  
          "camG":30,  
          "calçaM":40,  
          "calçaG": 60}
```

Escreva um código que percorra o dicionário, e some as quantidades totais de camiseta e de calça vendidos na loja.

```
vendas = {"camP":10,"calçaP":20,"camM":20,"camG":30,"calçaM":40,"calçaG": 60}  
calcas = 0  
camisetas = 0  
for chave in vendas:  
    if "cam" in chave:  
        camisetas += vendas[chave]  
    if "calça" in chave:  
        calcas += vendas[chave]  
  
print (f"total de calças vendidas: {calcas}")  
print (f"total de camisetas vendidas: {camisetas}")
```

2. Considere o seguinte dicionário, onde as chaves são nomes dos alunos e os valores são as notas obtidas por eles ao longo do semestre. Calcule e imprima a média de cada aluno.

```
notas = {"Ana": [9.0,8.5,8.0],  
         "João": [4.0,5.0,6.0],  
         "Maria": [5.0,6.0,7.0]}
```

```
notas = {"Ana": [9.0,8.5,8.0],  
         "João": [4.0,5.0,6.0],  
         "Maria": [5.0,6.0,7.0]}  
  
for aluno in notas:  
    media = sum(notas[aluno])/len(notas[aluno])  
    print(f"A média de {aluno} é: {media}")
```

3. Considere o seguinte dicionário, onde as chaves são os nomes dos vendedores e os valores são listas com os valores das vendas mensais de cada um deles.

```
vendas = {"Carlos": [1200, 1500, 1700],  
          "Aline": [1800, 2000, 1400],  
          "João": [2400, 3000, 1200]}  
  
for chave in vendas:  
    total_vendedor = sum(vendas[chave])  
    print(f"O total de vendas do vendedor {chave} foi {total_vendedor}")
```

## 9 Zip, desempacotamento e enumerate

### 9.1 Comando zip

O comando zip em Python serve para agrupar elementos de duas ou mais sequências (listas, tuplas, etc.).

**Exemplo 9.1:** O comando zip gera um elemento iterável

```
lista1 = [1,2,3]
lista2 = ["a","b","c"]

lista_combinada = zip(lista1,lista2)

print(lista_combinada)
```

Impressão / Saída:

```
<zip object at 0x0000021509E3BBC0>
```

**Exemplo 9.2:** Iteração com for no elemento iterável criado pelo comando zip

```
lista1 = [1,2,3]
lista2 = ["a","b","c"]

for item in zip(lista1,lista2):
    print(item)
```

Impressão / Saída:

```
(1, 'a')
(2, 'b')
(3, 'c')
```

**Exemplo 9.3:** Criação de uma lista a partir do zip

```
lista1 = [1,2,3]
lista2 = ["a","b","c"]

lista_combinada = list(zip(lista1,lista2))
print (lista_combinada)
```

Impressão / Saída:

```
[(1, 'a'), (2, 'b'), (3, 'c')]
```

**Exemplo 9.4:** Criação de um dicionário a partir do zip

```
lista1 = [1,2,3]
lista2 = ["a","b","c"]

dicionario_combinado = dict(zip(lista1,lista2))
print (dicionario_combinado)
```

Impressão / Saída:

```
{1: 'a', 2: 'b', 3: 'c'}
```

**Exemplo 9.5:** A lista criada a partir do zip tem tamanho da menor das listas agrupadas

```
lista1 = [1,2,3,4,5]
lista2 = ["a","b","c"]

lista_combinada = list(zip(lista1,lista2))

print(lista_combinada)
```

Impressão / Saída:

```
[(1, 'a'), (2, 'b'), (3, 'c')]
```

**Exemplo 9.6:**

```
nomes = ["João", "Ana", "Maria"]
idades = [25, 22, 30]
cidades = ["Palmas", "Belém", "Manaus"]

idd_cdd = list(zip(idades, cidades))

dados = dict(zip(nomes, idd_cdd))

print(dados)
```

Impressão / Saída:

```
{'João': (25, 'Palmas'), 'Ana': (22, 'Belém'), 'Maria': (30, 'Manaus')}
```

## 9.2 Desempacotamento de listas e tuplas

**Exemplo 9.7:** unpack (desempacotamento) de um string

```
nome = "banana"

for letra in nome:
    print(letra)
```

Impressão / Saída:

```
b
a
n
a
n
a
```

**Exemplo 9.8:** desempacotamento de uma lista

```
lista3 = [(1, 'a'), (2, 'b'), (3, 'c')]

for numero, letra in lista3:
    print(numero)
    print(letra)
    print("##")
```

Impressão / Saída:

```
1
a
##
2
b
##
3
c
##
```

**Exemplo 9.9:** desempacotamento de uma tupla

```
codigos = (["a", 11], ["b", 22], ["c", 33])

for dado1, dado2 in codigos:
    print(dado1)
    print(dado2)
    print("***")
```

Impressão / Saída:

```
a
11
**
b
22
**
c
33
**
```

### 9.3 Desempacotamento de dicionários

**Exemplo 9.10:** para desempacotar dicionários é necessário o uso do método .items()

```
dados = {"Ana": 8.0, "Beto": 7.5}

for nome, media in dados.items():
    print(f"Nome - {nome}; Média - {media}")
```

Impressão / Saída:

```
Nome - Ana; Média - 8.0
Nome - Beto; Média - 7.5
```

### 9.4 Comando enumerate

**Exemplo 9.11:** uso do enumerate em uma string

```
nome = "banana"

for i, letra in enumerate(nome):
    print(f"{i}: {letra}")
```

Impressão / Saída:

```
0: b
1: a
2: n
3: a
4: n
5: a
```

**Exemplo 9.12:** parâmetro start no comando enumerate

```
nome = "banana"

for i, letra in enumerate(nome, start=1):
    print(f"{i}ª letra: {letra}")
```

Impressão / Saída:

```
1ª letra: b
2ª letra: a
3ª letra: n
4ª letra: a
5ª letra: n
6ª letra: a
```

**Exemplo 9.13:**

```
nomes = ["João", "Maria", "Carlos", "Ana"]
idades = [25, 30, 22, 28]
cidades = ["Belém", "Palmas", "Manaus", "Curitiba"]

for nm, idd, cid in zip(nomes, idades, cidades):
    print(f"{nm} tem {idd} anos e mora em {cid}")
```

Impressão / Saída:

```
João tem 25 anos e mora em Belém
Maria tem 30 anos e mora em Palmas
Carlos tem 22 anos e mora em Manaus
Ana tem 28 anos e mora em Curitiba
```

**Exemplo 9.14:** enumerate em dicionários precisa ser em conjunto com o método .items()

```
dados = {"Ana": 8.0, "Beto": 7.5}

for item, (nome, media) in enumerate(dados.items()):
    print(f"{item} - Nome - {nome}; Média - {media}")
```

Impressão / Saída:

```
0 - Nome - Ana; Média - 8.0
1 - Nome - Beto; Média - 7.5
```



## 9.4 Exercícios propostos

1. Reescreva o seguinte código usando zip e unpack (desempacotamento):

```
nomes = ['Ana', 'Beto', 'Carla']
idades = [25, 30, 22]
for i in range(len(nomes)):
    print(nomes[i], idades[i])
```

2. Reescreva o seguinte código usando desempacotamento:

```
lista = [[1,2,3], [4,5,6], [7,8,9]]
for numeros in lista:
    print(numeros[1])
```

3. Reescreva o seguinte código usando desempacotamento e enumerate:

```
nome = "Python"
i = 0
while i < len(nome):
    print(f"{i+1}ª letra - {nome[i]}")
    i += 1
```

4. Escreva os seguintes códigos usando zip e desempacotamento de listas.

- a) Seja dada uma lista com CPF de clientes devedores, uma lista com valores devidos por cada um dos clientes e outra lista com os dias de atraso desse valor. Deverá ser criada uma nova lista com o CPF dos clientes devedores que a empresa considera como inadimplentes, sendo que será considerado inadimplente o cliente que tiver devendo mais que R\$ 2.000,00 por mais de 20 dias. Informe qual o percentual de clientes considerados inadimplentes em relação a quantidade de clientes devedores.

```
cpf_devedor = ["456.810.836-41", "783.613.218-67", "989.379.232-65", "748.663.779-88"]
valor_devido = [3192.50, 1960.34, 3447.43, 2584.36]
dias_devidos = [18, 28, 25, 34]
```

- b) Crie três funções distintas, de acordo com o solicitado nos itens a, b e c. Cada uma delas receberá como parâmetro uma lista contendo o registro diário dos clientes de uma loja de departamentos, conforme modelo a seguir:

```
clientes = [[cliente1, valor_compra1], [cliente2, valor_compra2], ...]:
```

- b.1) função que retorne quantos clientes compraram mais de R\$ 1.000,00 no dia
- b.2) função que retorne o valor total vendido no dia
- b.3) função que retorne uma string informando o valor da maior compra do dia e o nome do cliente que fez essa compra, no formato: "Valor da maior compra: xxx; cliente: xxxx"

- c) Escreva uma função que recebe três listas: nomes, notas1, notas2 e retorne um dicionário com o nome como chave e a média como valor.

## 9.5 Resolução dos exercícios propostos

### Exercício 1a:

```
nomes = ["Ana", "Beto", "Carla"]
idades = [25, 30, 22]

for nome, idade in zip(nomes, idades):
    print(nome, idade)
```

### Exercício 1b:

```
lista = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

for n1, n2, n3 in lista:
    print(n2)
```

### Exercício 2a:

```
cpf_devedor = ["456.810.836-41", "783.613.218-67", "989.379.232-65", "748.663.779-88"]
valor_devido = [3192.50, 1960.34, 3447.43, 2584.36]
dias_devidos = [18, 28, 25, 34]

dados = list(zip(cpf_devedor, valor_devido, dias_devidos))

inadimplentes = []

for cpf, valor, dia in dados:
    if valor > 2000 and dia > 20:
        inadimplentes.append(cpf)
print(inadimplentes)
```

### Exercício 2b:

```
def clientes_mais1000(clientes):
    c_mais1000 = 0
    for cliente, compra in clientes:
        if compra > 1000:
            c_mais1000 += 1
    return c_mais1000

def venda_total_dia(clientes):
    v_total = 0
    for cliente, compra in clientes:
        v_total += compra
    return v_total

def msg_maior_compra(clientes):
    maior_compra = 0
    for cliente, compra in clientes:
        if compra > maior_compra:
            maior_compra = compra
            maior_cliente = cliente
    return f"Valor da maior compra: {maior_compra}; cliente: {maior_cliente}"
```

### Exercício 2c:

```
def montar_dict(nomes, notas1, notas2):
    dic_notas = {}

    for nome, n1, n2 in zip(nomes, notas1, notas2):
        media = (n1 + n2) / 2
        dic_notas[nome] = media
    return dic_notas
```



## 10 Manipulando arquivos em Python

### 10.1 Arquivos .txt

Arquivos .txt são arquivos de texto simples. São usados para guardar frases, listas ou dados simples linha a linha.

Modos de abertura de arquivos:

“r” – leitura

“w” – escrita (sobrescreve)

“a” – append (acrescenta ao final)

“x” – cria novo arquivo (erro se já existir)

#### Exemplo 10.1:

```
arquivo = open("teste1.txt", "r")
conteudo = arquivo.read()
print (conteudo)
arquivo.close()
```

#### Exemplo 10.2:

```
arquivo = open("teste1.txt", "r", encoding="utf-8")
conteudo = arquivo.read()
print (conteudo)
arquivo.close()
```

#### Exemplo 10.3: Uso do readlines

```
arquivo = open("teste1.txt", "r", encoding="utf-8")
conteudo = arquivo.readlines()
print (conteudo)
arquivo.close()
```

#### Exemplo 10.4:

```
with open("teste1.txt", "r", encoding="utf-8") as arquivo:
    conteudo = arquivo.read()
    print(conteudo)
```

#### Exemplo 10.5:

```
with open("teste1.txt", "r", encoding="utf-8") as arquivo:
    contador = 0
    for linha in arquivo:
        contador += 1
    print(f"O arquivo tem {contador} linhas")
```

#### Exemplo 10.6:

```
with open("teste1.txt", "r", encoding="utf-8") as arquivo:
    contador = 0
    for linha in arquivo:
        print(linha)
        contador += 1
    print(f"O arquivo tem {contador} linhas")
```

**Exemplo 10.7:**

```
with open("teste1.txt", "r", encoding="utf-8") as arquivo:
    contador = 0
    for i,linha in enumerate(arquivo):
        print (i, linha)
        contador += 1
    print(f"O arquivo tem {contador} linhas")
```

**Exemplo 10.8:** Criando um arquivo .txt

```
with open("exemplo2.txt", "x", encoding="utf-8") as arquivo:
    arquivo.write("Linha 1: arquivo criado com 'x'\n")
```

**Exemplo 10.8:** Escrever (sobrescrever) um arquivo .txt

```
with open("exemplo2.txt", "w", encoding="utf-8") as arquivo:
    arquivo.write("Agora estou trocando o conteúdo do arquivo")
```

**Exemplo 10.9:** Adicionar dados a um arquivo .txt

```
with open("exemplo2.txt", "a", encoding="utf-8") as arquivo:
    arquivo.write("Acrescentando dados 1")
    arquivo.write("Acrescentando dados 2")
```

## 10.2 Exercício resolvido com manipulação de arquivo .txt

1. Você deve criar um arquivo de nome *notas.txt* onde serão armazenados os dados de alunos no seguinte formato:

código;nome;nota1;nota2;nota3

Ex: ES256;Ana Maria;9.0;5.6;8.5

Os dados deverão ser solicitados ao usuário, acrescentando uma nova linha com cada aluno até que o usuário informe que não deseja mais cadastrar novos alunos.

Após a entrada de dados, o cadastro feito deverá ser exibido na forma a seguir:

Código: ES256 – Nome: Ana Maria – Notas: 9.0,5.6,8.5 – Média – 7,7

Deverá ser criado um arquivo com os nomes dos alunos aprovados.

```

with open("notas.txt", "w", encoding="utf-8") as arquivo:
    while True:
        print ("Cadastro de alunos:\n")
        codigo = input("Código do aluno: ")
        nome = input("Nome do aluno: ")
        nota1 = float(input("Nota 1:"))
        nota2 = float(input("Nota 2:"))
        nota3 = float(input("Nota 3:"))
        linha = f"{codigo};{nome};{nota1};{nota2};{nota3}\n"
        arquivo.write(linha)
        continuar = input("Deseja continuar (S) ou (N)? ")
        if continuar.lower() != "s":
            break

print ("\nAlunos cadastrados:")
aprovados = []
with open("notas.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        dados = linha.strip().split(";")
        cod = dados[0]
        nome = dados[1]
        n1 = float(dados[2])
        n2 = float(dados[3])
        n3 = float(dados[4])
        media = (n1+n2+n3)/3
        print(f"Código:{cod} - Nome:{nome} - Notas:{n1},{n2},{n3}- Média:{media}")
        if media >= 7.0:
            aprovados.append(nome)

with open("aprovados.txt", "w", encoding = "utf-8") as aprovados_arquivo:
    for nome in aprovados:
        aprovados_arquivo.write(nome + "\n")

print ("Lista dos aprovados")
print (aprovados_arquivo)

```



```

def cadastrar_alunos(nome_arquivo):
    with open(nome_arquivo, "w", encoding="utf-8") as arquivo:
        while True:
            print ("Cadastro de alunos:\n")
            codigo = input("Código do aluno: ")
            nome = input("Nome do aluno: ")
            nota1 = float(input("Nota 1:"))
            nota2 = float(input("Nota 2:"))
            nota3 = float(input("Nota 3:"))
            linha = f"{codigo};{nome};{nota1};{nota2};{nota3}\n"
            arquivo.write(linha)
            continuar = input("Deseja continuar (S) ou (N)? ")
            if continuar.lower() != "s":
                break

def mostrar_alunos(nome_arquivo):
    print ("\nAlunos cadastrados:")
    aprovados = []
    with open(nome_arquivo, "r", encoding="utf-8") as arquivo:
        for linha in arquivo:
            dados = linha.strip().split(";")
            cod = dados[0]
            nome = dados[1]
            n1 = float(dados[2])
            n2 = float(dados[3])
            n3 = float(dados[4])
            media = (n1+n2+n3)/3
            print(f"Código:{cod} - Nome:{nome} - Notas:{n1},{n2},{n3}- Média:{media}")
            if media >= 7.0:
                aprovados.append(nome)
    return aprovados

def salvar_aprovados(nome_arquivo, lista_aprovados):
    with open(nome_arquivo, "w", encoding = "utf-8") as aprovados_arquivo:
        for nome in aprovados:
            aprovados_arquivo.write(nome + "\n")

arquivo_notas = "notas.txt"
arquivo_aprovados = "aprovados.txt"
cadastrar_alunos(arquivo_notas)
aprovados = mostrar_alunos(arquivo_notas)
salvar_aprovados(arquivo_aprovados, aprovados)

```

## 10.3 Exercícios propostos

1. Crie um script que:
  - a) Peça ao usuário para digitar 5 tarefas que ele pretende fazer hoje
  - b) Grave essas tarefas em um arquivo tarefas.txt, com uma tarefa por linha. O usuário deverá digitar somente a tarefa (ex. "Estudar Python", "Ir ao mercado",..., mas a gravação no arquivo deverá ficar do tipo:
    1. Estudar Python
    2. Ir ao mercado
    - 3 .....
2. Crie um script que leia o conteúdo gerado no arquivo tarefas.txt (a partir do script 1) e imprima cada tarefa em uma linha.
3. Escreva um script que leia o conteúdo gerado no arquivo tarefas.txt e pergunte ao usuário qual das tarefas ele quer que seja impressa e então imprima essa tarefa.
4. Crie um arquivo vendas.txt a partir de dados digitados pelo usuário, no seguinte formato:  
código;produto;preço\_unitário;quantidade  
A cada venda registrada, o programa deverá perguntar se deseja continuar cadastrando. O processo de cadastro das vendas termina quando o usuário responder que não deseja mais.  
Exemplo de linha cadastrada:  
P102;Camisa Polo;89.90;3
5. A partir do arquivo vendas.txt, faça a leitura do arquivo e exiba todas as vendas cadastradas no seguinte formato:  
Código: P102 – Produto: Camisa Polo – Preço Unitário: R\$ 89.90 – Qtde.: 3 – Total: R\$ 269.70
6. Crie um novo arquivo chamado vendas\_maiores.txt, contendo apenas os produtos cujo total da venda ultrapassou R\$ 200,00.

## 10.4 Resolução dos exercícios propostos

### Código 1:

```
str_tarefa = ""
for i in range(1,6):
    tarefa = input("Digite a tarefa: ")
    str_tarefa += str(i)+". "+tarefa+"\n"

with open("tarefas.txt", "w", encoding="utf-8") as arquivo:
    arquivo.write(str_tarefa)
```

### Código 2:

```
with open("tarefas.txt", "r", encoding = "utf-8") as arquivo:
    for tarefa in arquivo.readlines():
        print (tarefa)
```

### Código 3:

```
with open("tarefas.txt", "r", encoding = "utf-8") as arquivo:
    tarefas = arquivo.readlines()
opcao = int(input("Qual tarefa quer visualizar? "))
print(tarefas[opcao-1])
```

### Código 4:

```
with open("vendas.txt", "w", encoding="utf-8") as arquivo:
    while True:
        codigo = input("Digite o código do produto: ")
        produto = input("Digite o nome do produto: ")
        preco_unitario = float(input("Digite o preço unitário: "))
        quantidade = int(input("Digite a quantidade vendida: "))

        linha = f"{codigo};{produto};{preco_unitario:.2f};{quantidade}\n"
        arquivo.write(linha)

        continuar = input("Deseja cadastrar outra venda?(s/n) ").strip().lower()
        if continuar != "s":
            break
```

### Código 5:

```
with open("vendas.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        dados = linha.strip().split(";")
        codigo = dados[0]
        produto = dados[1]
        preco = float(dados[2])
        qtde = int(dados[3])
        total = preco * qtde

        print(f"Código:{codigo}-Produto:{produto}-P.Unit.: R${preco:.2f}-Qtde:{qtde}-Total:R${total:.2f}")
```

### Código 6:

```
with open("vendas.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        dados = linha.strip().split(";")
        total = float(dados[2])*float(dados[3])
        if total > 200:
            with open("vendas_maiores.txt", "a", encoding="utf-8") as arq:
                arq.write(dados[1]+"\n")
```

## 11 Complementos

### 11.1 Exemplos com while True + break

Código 8.1: Contador simples até 5

```
contador = 1
while True:
    print(contador)
    if contador == 5:
        break
    contador += 1
```

Impressão / Saída:

```
1
2
3
4
5
```

Código 8.2: Esperar o usuário digitar "sair"

```
while True:
    comando = input("Digite algo (ou 'sair' para encerrar): ")
    if comando == "sair":
        break
```

Impressão / Saída:

```
Digite algo (ou 'sair' para encerrar): ab
Digite algo (ou 'sair' para encerrar): cd
Digite algo (ou 'sair' para encerrar): ef
Digite algo (ou 'sair' para encerrar): sair
>>>|
```

Código 8.3: Validação de entrada numérica

```
while True:
    entrada = input("Digite um número: ")
    if entrada.isnumeric():
        numero = int(entrada)
        break
    print("Entrada inválida. Tente novamente.")
```

Impressão / Saída:

```
Digite um número: 3a
Entrada inválida. Tente novamente.
Digite um número: a56
Entrada inválida. Tente novamente.
Digite um número: 56
>>>|
```

#### Código 8.4: Validação de entrada numérica

```
soma = 0
while True:
    n = int(input("Digite um número (0 para parar): "))
    if n == 0:
        break
    soma += n
print("Soma total:", soma)
```

#### Código 8.4: Menu simples com opção de saída se escolher a opção 0

```
while True:
    print("\n1 - Opção A\n2 - Opção B\n0 - Sair")
    escolha = input("Escolha: ")
    if escolha == "0":
        break
    print(f"Você escolheu a opção {escolha}")
```

#### Impressão / Saída:

```
1 - Opção A
2 - Opção B
0 - Sair
Escolha: 1
Você escolheu a opção 1

1 - Opção A
2 - Opção B
0 - Sair
Escolha: 2
Você escolheu a opção 2

1 - Opção A
2 - Opção B
0 - Sair
Escolha: 0
>>>|
```

#### Código 8.5: Soma de números até o usuário digitar 0

```
soma = 0
while True:
    n = int(input("Digite um número (0 para parar): "))
    if n == 0:
        break
    soma += n
print("Soma total:", soma)
```

#### Impressão / Saída:

```
Digite um número (0 para parar): 2
Digite um número (0 para parar): 4
Digite um número (0 para parar): 5
Digite um número (0 para parar): 0
Soma total: 11
```



## Anexos

### A – Resolução dos exercícios propostos 1 (item 1.9)

#### Código 1:

```
nome = input("Qual o seu nome? ")
idade = int(input("Qual a sua idade? "))

print ("Olá,", nome, "! Você tem", idade, "anos.")
```

#### Código 2:

```
nome = input("Qual o seu nome? ")
profissao = input("Qual a sua profissao? ")

print ("Olá,", nome, "! Você trabalha como", profissao, ". Desejamos um ótimo dia!")
```

#### Código 3:

```
num1 = int(input("Digite o primeiro número: "))
num2 = int(input("Digite o segundo número: "))
print ("A soma dos números é: ", num1 + num2)
print ("A diferença entre os números é: ", num1 - num2)
print ("O produto dos números é: ", num1 * num2)
print ("O divisão de um pelo outro é: ", num1 / num2)
```

#### Código 4:

```
cels = float(input("Qual a temperatura em graus Celsius? "))

fahr = cels * 9/5 + 32
print (cels, "°C =", fahr, "°F")
```

#### Código 5:

```
numero = float(input("Qual o número? "))
if numero > 0:
    print ("O número", numero, "é positivo")
elif numero < 0:
    print ("O número", numero, "é negativo")
else:
    print ("o número digitado é zero")
```

#### Código 6:

```
numero = int(input("Qual o número? "))

if numero%2 == 0:
    print ("O número", numero, "é par")
else:
    print ("O número", numero, "é ímpar")
```

#### Código 7:

```
numero = int(input("Digite um número inteiro entre 5 e 50: "))

cont = 0
while cont <= numero:
    print (cont)
    cont = cont + 1
```



### Código 8:

```
numero = int(input("Digite um número inteiro entre 100 e 500: "))

cont = 1
while cont <= numero:
    if cont % 2 == 0:
        print (cont)
    cont = cont + 1
```

### Código 9:

```
temp = float(input("Digite a temperatura atual: "))
if temp < 15:
    print ("Frio")
elif temp <= 25:
    print ("Agradável")
else:
    print ("Calor")
```

### Código 10:

```
lista_nomes = []
i = 0
while i < 5:
    nome = input("Digite um nome: ")
    lista_nomes.append(nome)
    i = i + 1

i = 0
while i < 5:
    print (lista_nomes[i])
    i = i + 1
```

### Código 11:

```
produtos = []
i = 0
while i < 3:
    novo_produto = []
    produto = input("Digite o produto: ")
    preco = float(input("Qual o preço desse produto? "))
    novo_produto.append(produto)
    novo_produto.append(preco)
    produtos.append(novo_produto)
    i = i + 1

i = 0
while i < 3:
    print ("Produto:", produtos[i][0], "- Preço: R$", produtos[i][1])
    i = i + 1
```

### Código 12:

```
maior_sublista=0
i = 0
while i < len(letras):
    if len(letras[i]) > maior_sublista:
        maior_sublista = len(letras[i])
    i = i + 1

i = 0
while i < len(letras):
    while len(letras[i])<maior_sublista:
        letras[i].append("X")
    i = i + 1

print (letras)
```

## **B – Resolução dos exercícios propostos 2 (item 3.3)**

### **Código 1:**

```
lista = [['A', 'B', 'C', 'D', 'E'], ['P', 'Q', 'R', 'S'], ['X', 'Y', 'Z']]
print (f"lista original: {lista}")

for i in range(len(lista)):
    lista[i][len(lista[i])-1] = lista[i][0]
print (lista)
```

```
lista = [['A', 'B', 'C', 'D', 'E'], ['P', 'Q', 'R', 'S'], ['X', 'Y', 'Z']]
print (f"lista original: {lista}")

for sublista in lista:
    if len(sublista)>1:
        sublista[-1] = sublista[0]
print(lista)
```

### **Código 2:**

```
lista = ['Joao', 'Pedro', 'Zezé', 'Daniel', 'Ana', 'Maria', 'Carla']

lista.sort (reverse=True)

for nome in lista:
    print(nome)
```

### **Código 3:**

```
lista = ['Joao', 'Pedro', 'Zezé', 'Daniel', 'Ana', 'Maria', 'Carla']

nova_lista = []
for nome in lista:
    if nome != "Ana":
        nova_lista.append(nome)
print (nova_lista)
```

### **Código 4:**

```
lista = ['Joao', 'Pedro', 'Zezé', 'Daniel', 'Ana', 'Maria', 'Carla']

for nome in lista:
    if nome == "Ana":
        lista.remove(nome)
print (lista)
```

### **Código 5:**

```
lista=[['A', 'B', 'C'] , ['I', 'J'], ['P', 'Q', 'R', 'S', 'T'], ['X', 'Y', 'Z']]
tam_m_sublista = 0

for sublista in lista:
    if len(sublista)> tam_m_sublista:
        tam_m_sublista = len(sublista)

for sublista in lista:
    while len(sublista)<tam_m_sublista:
        sublista.append("X")

print (lista)
```

## C – Resolução dos exercícios propostos 3 (item 4.4)

### Código 1:

```
import random

matriz = []
for i in range(4):
    nova_linha = []
    for j in range(4):
        nova_linha.append(random.randint(1,10))
    matriz.append(nova_linha)

str_matriz = ""
cont_pares = 0
cont_impares = 0
soma = 0

for i in range(4):
    for j in range(4):
        soma += matriz[i][j]
        str_matriz += str(matriz[i][j]) + "\t"
        if matriz[i][j] % 2 == 0:
            cont_pares += 1
        else:
            cont_impares += 1
    str_matriz += "\n"
media = soma/16

print ("A matriz gerada foi:\n" + str_matriz)
print (f"A soma de todos os elementos dessa matriz é: {soma}")
print (f"Há {cont_pares} números pares e {cont_impares} números ímpares na matriz")
```

### Código 2:

```
import random

linhas = int(input("Quantas linhas terá a sua matriz? "))
colunas = int(input("Quantas colunas terá a sua matriz? "))
matriz = []
for i in range(linhas):
    nova_linha = []
    for j in range(colunas):
        nova_linha.append(random.randint(1,10))
    matriz.append(nova_linha)

for linha in matriz:
    print (linha)

alterar_linha = int(input("Qual linha deseja alterar? "))
alterar_coluna = int(input("Qual coluna deseja alterar? "))
novo_valor = int(input("Qual o novo valor para esse elemento? "))

if 0 <= alterar_linha < linhas and 0 <= alterar_coluna < colunas:
    matriz[alterar_linha][alterar_coluna] = novo_valor
else:
    print ("Posição inválida")

for linha in matriz:
    print (linha)
```

## **D – Resolução dos exercícios propostos 4 (item 5.2)**

### **Código 1:**

```
import random

lista = []
for i in range (5):
    lista.append(random.randint(0,1000))
print (f"Lista original: {lista}")

for i in range(3):
    num = int(input("Qual número você deseja adicionar a essa lista? "))
    pos = int(input(f"Em qual posição você deseja adicionar o número {num}? "))
    lista.insert(pos, num)
print (f"Lista com elementos inseridos: {lista}")

lista.sort(reverse=True)
print (f"Lista com elementos em ordem decrescente: {lista}")
```

### **Código 2:**

```
frutas = ["manga", "amora", "laranja", "banana", "melancia"]

nome_fruta = input("Digite o nome de uma fruta: ")

if nome_fruta in frutas:
    print(f"A fruta {nome_fruta} está na lista")
else:
    print(f"A fruta {nome_fruta} não está na lista")
```

### **Código 3:**

```
import random

lista = []
for i in range (10):
    lista.append(random.randint(0,100))
print (f"Lista original: {lista}")

lista_impares = []
for i in range(10):
    if lista[i] % 2 != 0:
        lista_impares.append(lista[i])
print (f"Lista com somente os elementos ímpares: {lista_impares}")
```



## E – Resolução dos exercícios propostos 5 (item 6.5)

### Código 1:

```
texto_sem_vogais = ""
texto = input("Digite uma frase: ")
vogais = "aeiouAEIOU"
for letra in texto:
    if letra not in vogais:
        texto_sem_vogais += letra
print("Texto sem vogais:", texto_sem_vogais)
```

Resolva também com as seguintes variações: insira espaço na string vogais; deixe somente as vogais minúsculas e faça a comparação no for letra in texto adicionando o método .lower() em texto; depois deixe somente as vogais maiúsculas e adicione o método .upper()

### Código 2:

```
cpf = input('Insira seu CPF')

if len(cpf) == 11 and cpf.isnumeric():
    print(cpf)
else:
    print('Digite seu CPF corretamente e digite apenas números')

cpf = input('Insira seu CPF')
cpf_valido = True

if len(cpf) != 11:
    print("Seu CPF não tem 11 dígitos")
    cpf_valido = False
if not cpf.isnumeric():
    print("Seu CPF tem caracteres não numéricos")
    cpf_valido = False

if cpf_valido:
    print(f"CPF {cpf} digitado corretamente")
else:
    print("Digite seu CPF novamente")
```

### Código 3:

```
cpf = input ("Insira seu CPF: ")

cpf = cpf.strip()
cpf = cpf.replace ('.', '')
cpf = cpf.replace ('-', '')

if len(cpf) == 11 and cpf.isnumeric():
    print ("Números do seu CPF: ", cpf)
else:
    print ("Digite seu CPF corretamente e digite apenas números")
```

### Código 4:

```
clientes_devedores=[['456.810.836-41', 3192.50, 18],
                    ['783.613.218-67', 3447.43, 25],
                    ['989.379.232-65', 2584.36, 34],
                    ['748.663.779-88', 2487.45, 38],
                    ['264.760.944-20', 1960.34, 28]]

inadimplentes=[]
for cliente in clientes_devedores:
    cpf = cliente[0]
    valor = cliente[1]
    dias = cliente [2]
    if valor >= 2000 and dias >=20:
        inadimplentes.append(cpf)
print(inadimplentes)
```

Nesse código acima, cada elemento da lista cliente é acessado individualmente por meio de seu índice (cliente[0], cliente[1] e cliente[2]).

```
inadimplentes=[]
for cliente in clientes_devedores:
    cpf, valor, dias = cliente
    if valor >= 2000 and dias >=20:
        inadimplentes.append(cpf)
print(inadimplentes)
```

Neste trecho do código acima, a lista cliente é **desempacotada diretamente** em três variáveis (cpf, valor e dias).

#### Código 5:

```
nome = input("Digite seu nome completo: ")
nome = nome.strip()

nome_valido = True
nome_formatado = ""
for palavra in nome.split():
    if palavra.isalpha():
        nome_formatado += palavra.capitalize() + " "
    else:
        nome_valido = False

if nome_valido:
    print(nome_formatado.strip())
else:
    print("Erro: O nome deve conter apenas letras.")
```

#### Código 6:

```
frase = input("Digite uma frase: ")

letras = 0
numeros = 0

for caractere in frase:
    if caractere.isalpha():
        letras += 1
    elif caractere.isnumeric():
        numeros += 1

print(f"Total de letras: {letras}")
print(f"Total de números: {numeros}")
```

#### Código 7:

```
placa = input("Digite a placa: ").strip()

if len(placa) == 8 and placa[:3].isalpha() and placa[3] == "-" and placa[4:].isnumeric():
    print(f"A placa {placa} é do modelo antigo")
elif len(placa) == 7 and placa[:3].isalpha() and placa[3].isnumeric() and placa[4].isalpha() and placa[5:].isnumeric():
    print(f"A placa {placa} é do modelo Mercosul")
else:
    print(f"A placa {placa} é inválida")
```



```
placa = input("Digite a placa: ")
placa = placa.strip()

placa_valida = True
if len(placa) == 8:
    if placa[0:3].isalpha() and placa[3] == "-" and placa[4:].isnumeric():
        print(f"A placa {placa} é do modelo antigo")
    else:
        placa_valida = False
elif len(placa) == 7:
    if placa[0:3].isalpha() and placa[3].isnumeric() and placa[4].isalpha() and placa[5:].isnumeric():
        print(f"A placa {placa} é do modelo Mercosul")
    else:
        placa_valida = False
else:
    placa_valida = False

if not placa_valida:
    print(f"A placa {placa} é inválida")
```

## F – Códigos diversos de revisão para G1

1. Uma empresa planejou suas campanhas de marketing para o ano, incluindo a "Campanha Primavera", "Campanha Verão", entre outras. Cada campanha possui um valor de investimento e uma estimativa de alcance em número de pessoas.

Os dados estão organizados na lista a seguir, onde cada item contém:

- Nome da campanha
- Valor do investimento (em reais)
- Alcance estimado

```
marketing = [['Campanha Primavera', 4500, 700], ['Campanha Verão', 6200, 550],  
             ['Campanha Outono', 5200, 600], ['Campanha Inverno', 3000, 300],  
             ['Campanha Black Friday', 8000, 800]]
```

A empresa considera uma *campanha de alto impacto* aquela com investimento superior a R\$ 5.000,00 e alcance estimado acima de 500 mil pessoas.

Escreva um código que:

- a) Crie e imprima uma nova lista contendo apenas os nomes das campanhas de alto impacto.
- b) Calcule e exiba o percentual de campanhas de alto impacto em relação ao total de campanhas planejadas.

```
campanhas_marketing = [['Campanha Primavera', 4500, 400],  
                       ['Campanha Verão', 6200, 550],  
                       ['Campanha Outono', 5200, 600],  
                       ['Campanha Inverno', 3000, 300],  
                       ['Campanha Black Friday', 8000, 800]]  
  
campanhas_alto_impacto = []  
  
for campanha in campanhas_marketing:  
    if campanha[1] > 5000 and campanha[2] > 500:  
        campanhas_alto_impacto.append(campanha[0])  
  
percentual_alto_impacto=(len(campanhas_alto_impacto)/len(campanhas_marketing))*100  
  
print("Campanhas de alto impacto:", campanhas_alto_impacto)  
print(f"Percentual de campanhas de alto impacto: {percentual_alto_impacto:.2f}%")
```

2. Em uma indústria, os horários de trabalho de limpeza são organizados com um código formado por um número de 1 a 7, representando o dia da semana.

Utilize a seguinte lista para associar os números aos dias correspondentes:

```
dias_semana = ["domingo", "segunda", "terça", "quarta",  
               "quinta", "sexta", "sábado"]
```

Solicite ao usuário um código numérico de 1 a 7 e exiba a mensagem:

**Seu próximo horário é <dia da semana>**

Exemplos:

Entrada: 2 Saída: **Seu próximo horário é segunda.**

Entrada: 5 Saída: **Seu próximo horário é quinta.**

*Obs: A resolução deve ser feita utilizando apenas um comando print.*

```

dias_semana = ["domingo", "segunda", "terça", "quarta",
               "quinta", "sexta", "sábado"]

dia = int(input("Qual o código do seu dia de trabalho? "))

if 0 <= dia <= 7:
    print (f"Seu próximo horário é {dias_semana[dia-1]}.")
else:
    print ("Código inválido para o dia")

```

3. Uma instituição deseja implementar um sistema de validação de senha para controlar o nível de acesso dos alunos. A senha digitada deve ter exatamente 6 caracteres e determinará o tipo de acesso concedido, conforme as seguintes regras:

- a) Se a senha contiver apenas letras (sem números), o acesso será "Básico".
- b) Se a senha contiver apenas números (sem letras), o acesso será "Intermediário".
- c) Se a senha for composta por duas letras seguidas de quatro números, o acesso será "Avançado".
- d) Qualquer outra combinação será considerada inválida.

Escreva um código que:

- i. Solicite ao usuário que digite uma senha
- ii. Determine o nível de acesso conforme as regras acima e exiba a mensagem correspondente ao nível de acesso ou informe que a senha é válida.

\* Lembre-se que o usuário pode, por engano, digitar espaços vazios antes ou depois dos caracteres do código de acesso. É necessário que esse erro seja tratado.

\*Use os métodos `.strip()`, `.isnumeric()` e `.isalpha()` para resolver essa questão.

Teste o seu código considere os seguintes exemplos de entrada de dados e confirme a saída:

|                          |                                    |
|--------------------------|------------------------------------|
| Entrada: <b>AHNTGR</b>   | Saída: <b>acesso básico</b>        |
| Entrada: <b>gftre</b>    | Saída: <b>acesso básico</b>        |
| Entrada: <b>235678</b>   | Saída: <b>acesso intermediário</b> |
| Entrada: <b>RT1976</b>   | Saída: <b>acesso avançado</b>      |
| Entrada: <b>RTGH21</b>   | Saída: <b>senha inválida</b>       |
| Entrada: <b>12345678</b> | Saída: <b>senha inválida</b>       |

```

senha = input("Digite a senha").strip()

if len(senha) == 6:
    if senha.isalpha():
        print("acesso básico")
    elif senha.isnumeric():
        print("acesso intermediário")
    elif senha[:2].isalpha() and senha[2:].isnumeric():
        print("acesso avançado")
    else:
        print("senha inválida")
else:
    print ("senha inválida")

```

4. Dada a seguinte lista de funcionários e seus respectivos salários:

```
funcionarios = [ ["2345", 2500], ["3467", 3200], ["5678", 4500],  
                 ["0234", 2200], ["1455", 3000], ["9654", 2800] ]
```

a) Exiba os dados de cada funcionário no seguinte formato:

**Funcionário:** [código] – **Salário:** R\$ [salário]

b) Identifique o funcionário com maior salário e exiba uma mensagem informando seu código.

c) Aplique um reajuste de 5% a todos os salários e armazene os novos valores em uma nova lista.

d) Exiba os dados dos funcionários com os salários reajustados no formato:

**Funcionário:** [código] – **Salário reajustado:** R\$ [salário]

```
funcionarios = [ ["2345", 2500], ["3467", 3200], ["5678", 4500],  
                 ["0234", 2200], ["1455", 3000], ["9654", 2800] ]  
  
maior_salario = 0  
novos_salarios = []  
  
for funcionario in funcionarios:  
    print(f"Funcionário: {funcionario[0]} – Salário: R$ {funcionario[1]}")  
    if funcionario[1] > maior_salario:  
        maior_salario = funcionario[1]  
        maior_func = funcionario[0]  
    novos_salarios.append([funcionario[0], funcionario[1]*1.05])  
  
print(f"O maior salário é: R${maior_salario}, e o funcionário é o {maior_func}")  
  
for func in novos_salarios:  
    print(f"Funcionário: {func[0]} – Salário: R$ {func[1]}")
```

5. Escreva um código que leia a data de nascimento de uma pessoa (dia, mês (1 a 12) e ano separadamente) e imprima a data com o mês escrito por extenso.

Considere o seguinte exemplo de entrada de dados:

Qual o dia do seu nascimento? 14

Qual o mês do seu nascimento? 03

Qual o ano do seu nascimento? 1983

Saída dos dados:

Você nasceu em 14 de março de 1983.

**Use a lista:**

```
meses = ["janeiro", "fevereiro", "março", "abril", "maio", "junho", "julho", "agosto",  
         "setembro", "outubro", "novembro", "dezembro"]
```

*Não precisa validar as entradas; considere que o usuário digitará valores corretos para os dias (1 a 31), para os meses (1 a 12) e para um ano válido.*

```

dia = input("Qual o dia do seu nascimento? ")
mes = int(input("Qual o mês do seu nascimento? "))
ano = input("Qual o ano do seu nascimento? ")

meses = ["janeiro", "fevereiro", "março", "abril", "maio", "junho",
         "julho", "agosto", "setembro", "outubro", "novembro", "dezembro"]

print (f"Você nasceu em {dia} de {meses[mes-1]} de {ano}.")

```

6. Uma faculdade de computação vai promover um congresso onde o código de acesso é gerado na hora da inscrição, conforme o participante seja de um dos seguintes grupos: aluno ou egresso da instituição, ou ainda aluno de uma outra instituição.

O modelo do código de acesso segue os seguintes padrões:

Aluno da instituição: ABC-12D3

Egresso da instituição: AB-12

Outros participantes: ABCD12

*Nos padrões indicados, onde aparece “AB” por exemplo, indica-se que são letras maiúsculas e onde aparece “12”, indicam-se números. Ou seja, no caso de aluno da instituição, serão três letras maiúsculas seguidas de um traço, dois números, uma letra e um número; Se egresso, serão duas letras maiúsculas, um traço e dois números,...*

Faça um código que leia o código de acesso e exiba as seguintes mensagens:

Se for um aluno da instituição: “Querido aluno, seja bem-vindo!”

Se for um egresso da instituição: “Muito bom revê-lo na nossa faculdade”

Se for um participante externo: “Seja bem-vindo ao nosso Congresso”

Se o código for inválido, escreva: “Código inválido”

\* Lembre-se que o usuário pode, por engano, digitar espaços vazios antes ou depois dos caracteres do código de acesso. É necessário que esse erro seja tratado.

\*Use os métodos `.strip()`, `.isnumeric()` e `.isalpha()` para resolver essa questão.

Considere os seguintes exemplos de entrada e saída:

Entrada: XEF-23R4 Saída: “Querido aluno, seja bem-vindo!”

Entrada: CR-46 Saída: “Muito bom revê-lo na nossa faculdade”

Entrada: DFRE24 Saída: “Seja bem-vindo ao nosso Congresso”

Entrada: RF345 Saída: “Código inválido”

Entrada: WERT4578 Saída: “Código inválido”

```
codigo = input("Digite o código: ").strip()

if (len(codigo)==8 and codigo[:3].isalpha()
    and codigo[3]=="-" and codigo[4:6].isnumeric()
    and codigo[6].isalpha() and codigo[7].isnumeric()):
    print ("Querido aluno, seja bem-vindo!")
elif (len(codigo)==5 and codigo[:2].isalpha()
    and codigo[2]=="-" and codigo[4:].isnumeric()):
    print ("Muito bom revê-lo na nossa faculdade")
elif len(codigo)==6 and codigo[:4].isalpha() and codigo[4:].isnumeric():
    print ("Seja bem-vindo ao nosso Congresso")
else:
    print ("Código inválido")
```



## G – Códigos diversos – usos de funções

1. Elabore um programa em Python que permita o cadastro de um número indeterminado de alunos, solicitando ao usuário as seguintes informações para cada aluno:
  - nome completo do estudante
  - ano de ingresso no curso (formato numérico de 4 dígitos)
  - nome do curso (por extenso)

\* A entrada de dados deve ser encerrada quando o usuário digitar “fim” para o nome do aluno.

Crie uma função chamada gerar\_matricula(), que receba o nome do aluno, o ano de ingresso e o curso e retorne uma string no seguinte formato:

- as 3 primeiras letras do curso em maiúsculas;
- os dois últimos dígitos do ano de ingresso;
- as duas primeiras letras do nome do aluno, em minúsculas
- dois números aleatórios entre 0 e 9 (formando os dois dígitos finais).

Ex: Para o curso de Engenharia de Software, ano de ingresso 2022, e aluno Lucas Silva, a matrícula poderá ser: ENG22lu57

Todos os dados deverão ser armazenados em uma lista de sublistas, onde cada sublista contém: [matrícula, nome do aluno, ano de ingresso, nome do curso]

Ao final, o programa deverá imprimir:

- a) a lista gerada com os dados de matrícula
- b) as seguintes informações de cada aluno cadastrado no seguinte formato:

Aluno: Lucas Silva; Matrícula: ENG22lu57

```
import random

def gerar_matricula(nome, ano, curso) :
    parte1 = curso.upper()[:3]
    parte2 = ano[:2]
    parte3 = nome.lower()[:2]
    parte4 = str(random.randint(0,9))+str(random.randint(0,9))
    return parte1+parte2+parte3+parte4

alunos = []
while True:
    nome = input("Qual o nome do aluno? ")
    if nome == "fim":
        break
    ano = input("Qual o ano de ingresso do aluno? ")
    curso = input("Qual o curso do aluno? ")
    matricula = gerar_matricula(nome,ano,curso)
    alunos.append([matricula,nome,ano,curso])

print ("Lista dos dados da matrícula")
print (alunos)

for aluno in alunos:
    print (f"Aluno: {aluno[1]}; Matrícula: {aluno[0]}")
```

2. Crie três funções distintas, de acordo com o solicitado nos itens a, b e c. Cada uma delas receberá como parâmetro uma lista contendo o registro diário dos clientes de uma loja de departamentos, conforme modelo a seguir:

```
clientes = [[cliente1, valor_compra1], [cliente2, valor_compra2],...]:
```

- a) função que retorne quantos clientes compraram mais de R\$ 1.000,00 no dia

```
def comprou_mais_1000(clientes):
    mais_1000 = 0
    for cliente in clientes:
        if cliente[1] > 1000:
            mais_1000 += 1
    return mais_1000
```

- b) função que retorne o valor total vendido no dia

```
def valor_total_vendido(clientes):
    valor_total = 0
    for cliente in clientes:
        valor_total += cliente[1]
    return valor_total
```

- c) função que retorne uma string informando o valor da maior compra do dia e o nome do cliente que fez essa compra, no formato: "Valor da maior compra: xxx; cliente: xxxx"

```
def maior_compra_dia(clientes):
    maior_compra = 0
    for cliente in clientes:
        if cliente[1] > maior_compra:
            maior_compra = cliente[1]
            maior_cliente = cliente[0]
    return f"Valor da maior compra: {maior_compra}; cliente: {maior_cliente}"
```

3. Considere a seguinte lista contendo o nome e o ano de fundação de algumas cidades do estado do Tocantins:

```
cidades= [ ["Paraíso", 1963 ], ["Gurupi", 1958], ["Palmas", 1989], ["Miranorte", 1964] ]
```

Você deverá criar um programa em Python que ofereça **um menu interativo com as opções descritas abaixo**. Para isso, será necessário implementar três funções conforme orientações a seguir:

- a) Função **imprimir\_cidade ()**

Esta função deve imprimir todas as cidades da lista no seguinte formato:

Cidade: Paraíso; Ano de criação: 1963

Cidade: Gurupi; Ano de criação: 1958

....

- b) Função **verificar\_data ()**

Esta função deve receber como parâmetro um ano fornecido pelo usuário, e verificar, para cada cidade da lista, se ela foi criada antes, depois ou exatamente naquele ano. A função deverá imprimir:

A cidade de Paraíso foi criada depois de 1960

A cidade de Gurupi foi criada antes de 1960

...

c) Função **acrescentar\_cidade()**

Esta função deve solicitar ao usuário que digite o nome de uma nova cidade e o ano sua criação, e em seguida, deve acrescentar essas informações à lista original.

Seu programa principal deve exibir o seguinte menu para o usuário:

Opção 1: Imprimir as listas das cidades

Opção 2: Verificar se as cidades foram criadas antes ou depois de um determinado ano.

Opção 3: Adicionar nova cidade à lista

Opção 4: Sair do programa

```
def verificar_data(ano):
    for cidade in cidades:
        if cidade[1] < ano:
            print(f"A cidade de {cidade[0]} foi criada antes de {ano}")
        elif cidade[1] > ano:
            print(f"A cidade de {cidade[0]} foi criada depois de {ano}")
        else:
            print(f"A cidade de {cidade[0]} foi criada em {ano}")

def acrescentar_cidade():
    nova_cidade = input("Qual cidade quer inserir? ")
    ano_criacao = int(input("Qual o ano da criação? "))
    cidades.append([nova_cidade, ano_criacao])

cidades=[["Paraíso", 1963 ], ["Gurupi", 1958],
          ["Palmas", 1989], ["Miranorte", 1964]]

print ('''
(1) - imprimir cidade
(2) - verificar data
(3) - acrescentar cidade
(4) - sair do programa''')

while True:
    opcao = input("Digite a opção desejada: ")
    if opcao == "1":
        imprimir_cidade()
    elif opcao == "2":
        verificar_data(1989)
    elif opcao == "3":
        acrescentar_cidade()
    else:
        break
```